影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 8 章 色彩影像處理

本章學習目標

- 說明三原色理論與人眼感色機制，理解 RGB 加色與 CMY 減色系統的差異。
- 描述 HSL 與 HSV 色彩模型的圓柱、圓錐與雙圓錐幾何詮釋，並說明兩者的差異。
- 比較 YCbCr 與 CIE Lab 色彩空間的設計目的與應用場合。
- 正確使用 OpenCV 進行色彩空間轉換，避開 BGR 與 HSV 數值範圍的陷阱。
- 運用虛擬色彩將灰階或非可見光資料轉為易於判讀的彩色影像，並理解色階選擇的專業考量。
- 以灰階世界法等演算法實作白平衡，並完成基於 HSV 的顏色分割與物件追蹤。
- 理解色彩恆常性對機器視覺的啟示，並據以設計不受光源干擾的檢測系統。

叫獸開講|藍黑還是白金?一件洋裝如何撕裂全世界

2015 年 2 月，一張洋裝照片被貼上社群網站，附帶一句求助：「這件洋裝到底是白金色還是藍黑色?」接下來的 48 小時，這個問題席捲全球，朋友吵架、情侶決裂、名人與政治人物紛紛表態，連視覺科學家都被捲進戰局。最詭異的是：雙方都不是在說謊，也不是視力有問題——他們看著同一堆像素，大腦卻給出截然不同的答案。而這件洋裝的真實顏色是藍色與黑色。

科學界的反應非常迅速。同年，麻省理工學院的 Rosa Lafer-Sousa、Katherine Hermann 與 Bevil Conway 在《Current Biology》發表了研究：他們對 1401 人做了自由回答(而非二選一)的調查，確認多數人確實只看到白金或藍黑

兩種答案，少數人看到藍棕;而且回報白金的比例在年長者與女性中較高。這證實了「分歧是真實的」，不是社群媒體提問方式造成的假象。

關鍵詞叫做色彩恆常性(Color Constancy)：同一顆蘋果，在正午陽光下、在黃昏夕照下、在螢光燈下，反射到你眼睛的光譜其實差異極大，但你始終覺得它是紅色的。這是因為大腦會自動推測「現在的光源是什麼顏色」，再把光源的影響扣掉，還原出物體本身的顏色——這就是人腦內建的自動白平衡。

問題在於，這張洋裝的照片曝光過度、背景線索模糊，光源資訊變得極度曖昧，大腦只能仰賴各自的「先驗假設」來猜。研究團隊做了一個漂亮的驗證：把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就會趨於一致，並且與被暗示的光源方向吻合。換句話說——看到白金的人，大腦假設場景被偏藍的日光或陰影照亮，於是扣掉藍色成分;看到藍黑的人，大腦假設是偏黃的室內人造光，於是扣掉黃色成分。

那為什麼不同人的假設不同?一個廣受討論的假說是生活作息：紐約大學的 Pascal Wallisch 在 2017 年的《Journal of Vision》發表了一項超過一萬三千人的大規模研究，發現自認是「雲雀」(早起、白天活動)的人較常看到白金，而「夜貓子」(習慣人造光下活動)的人較常看到藍黑，且呈現劑量反應的關係。不過也要誠實說明：Lafer-Sousa 團隊自己的資料對這個假說只提供了微弱的支持，學界至今仍在討論確切的機制。這也是科學該有的樣子——一個引人入勝的假說，需要更多證據來檢驗。

這個全民事件對影像處理學生有三重啟示。第一，你「看到」的顏色不是物理量，而是大腦處理過的結果——同一組 RGB 數值，可以被解讀成完全不同的顏色。第二，電腦視覺系統必須做同一件事：本章 8.5 節的白平衡演算法，就是在教機器扣掉光源的影響，而 Night Sight 的學習式白平衡(第 5 章)正是這條路線的現代版本。第三，也是最重要的：在工業檢測與機器人視覺中，如果光源不穩定，顏色判讀就會像這件洋裝一樣不可靠。這正是第 17 章 AOI 系統要不惜血本控制打

光的原因。

思考題：如果要設計一台「絕不會被光源騙倒」的顏色檢測系統，你會採取什麼策略?(提示一：控制光源;提示二：在畫面中放一張已知顏色的參考色卡;提示三：改用對光源較不敏感的特徵)

8.1 色彩的基礎：從人眼到 RGB

8.1.1 三原色理論

人眼視網膜有三種錐狀細胞，分別對長波(紅)、中波(綠)、短波(藍)敏感。任何顏色的光進入眼睛，都被化約成三個數值的組合送進大腦——這就是三原色理論的生理基礎，也是為什麼數位影像用三個通道就能表現幾乎所有顏色。

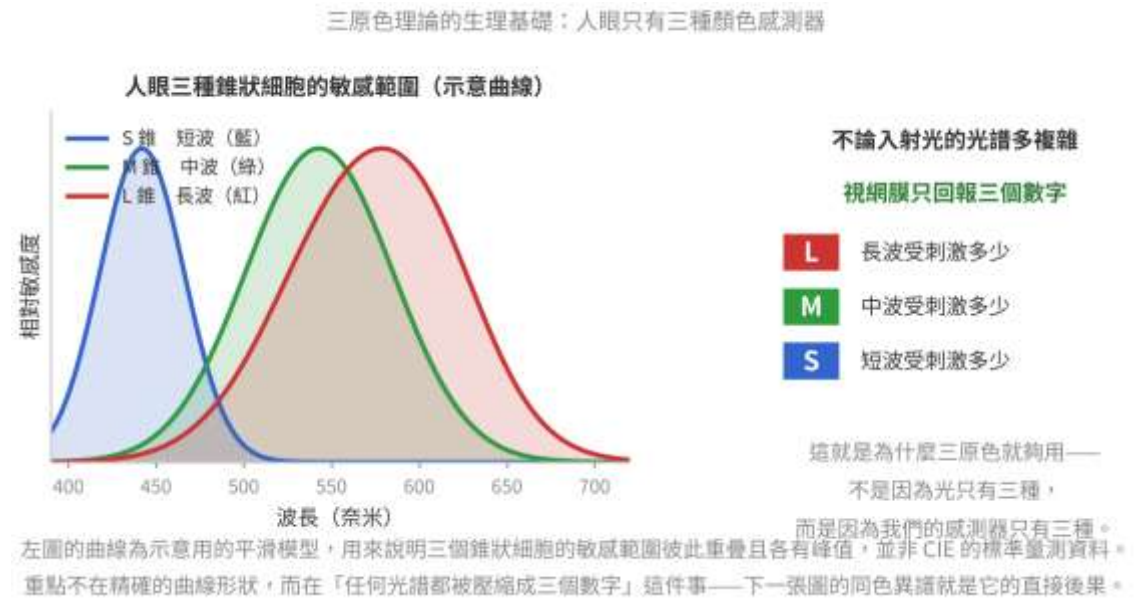


圖 8-1　三種錐狀細胞的敏感範圍：任何光譜都被壓成三個數字

值得深思的是同色異譜(Metamerism)現象：兩束光譜完全不同的光，只要刺激三種錐狀細胞的比例相同，人眼就會覺得它們是同一個顏色。這既是彩色顯示技術得以成立的原因(螢幕只用紅綠藍三種光源，就能模擬幾乎所有顏色)，也是色彩管理困難的根源——在某個光源下看起來相同的兩塊布料，換到另一個光源下可能顏色明顯不同，這在紡織與印刷業是實際的品管難題。

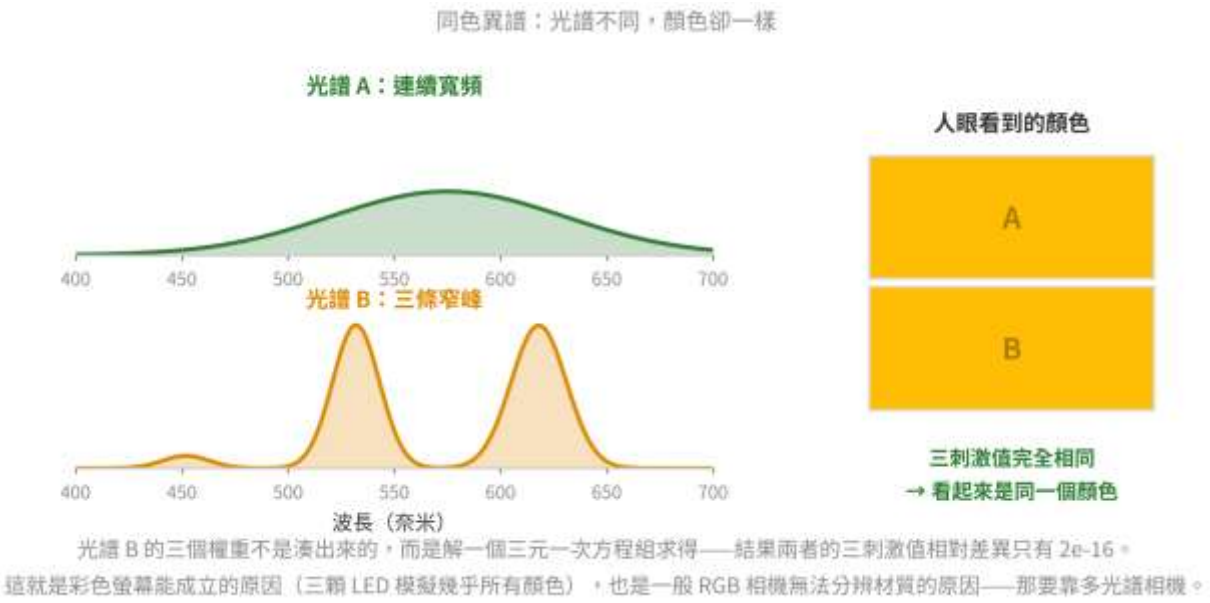


圖 8-2　同色異譜：光譜完全不同，三刺激值卻一模一樣

這也對機器視覺有一個重要啟示：一般 RGB 相機與人眼一樣，只記錄三個數值，因此無法分辨同色異譜的物體。若要真正區分材質(例如判斷兩塊看似同色的塑膠是否為同一種材料)，就需要記錄完整光譜的多光譜或高光譜相機——這在農產品分級、材料檢測、遙測中相當常見。

8.1.2 加色與減色系統

RGB 是加色系統：光源相加，紅加綠得黃，三色全開得白，適用於自發光的裝置——螢幕、投影機、LED。CMY(青、洋紅、黃)是減色系統：顏料吸收特定波長，疊愈多愈暗，三色相加理論上得黑(實務上因顏料不純而加入黑色 K，成為印刷用的 CMYK)。



圖 8-3　加色系統與減色系統：疊愈多愈亮，還是疊愈多愈暗

同一張照片在螢幕上鮮豔、印出來卻黯淡，正是因為兩個系統的色域(可表現的顏色範圍)不同。這也解釋了為什麼專業的印刷流程需要色彩管理與打樣——這正是 8.3.2 節 CIE Lab 存在的理由。

8.1.3 RGB 的侷限

RGB 對機器友善(直接對應感光元件與螢幕)，卻對人類不直觀。兩個具體的痛點：

第一，調整困難。請問「把這個顏色變深一點但色相不變」該怎麼改 RGB 三個數值?答案是三個都要按比例調整，很不方便;若只改其中一個，色相就跑掉了。

第二，也是對機器視覺更致命的：同一個物體在陰影中與陽光下，RGB 三個值全部大幅改變，難以判斷「還是不是同一個顏色」。舉例來說，一顆紅色蘋果在陽光下可能是 (220, 40, 40)，在陰影中變成 (90, 15, 15)——數值差了兩倍以上，但它顯然是同一顆蘋果。這使得「用 RGB 數值範圍來偵測特定顏色」在光線變化的環境中極不可靠。

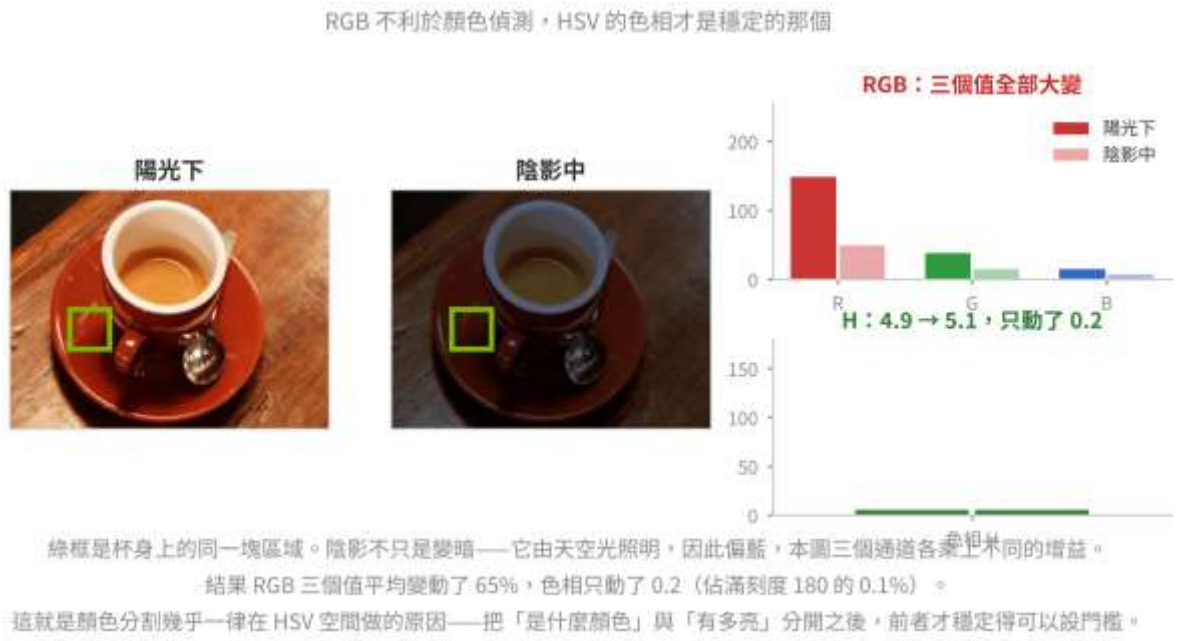


圖 8-4　同一顆蘋果進了陰影：RGB 全變，色相幾乎不動

這兩個痛點，催生了下一節的 HSL 與 HSV。

8.2 HSL 與 HSV：符合人類直覺的色彩模型

HSV(色相 Hue、飽和度 Saturation、明度 Value)與 HSL(色相、飽和度、亮度 Lightness)把「是什麼顏色」與「有多亮、多鮮豔」分離開來，更貼近人類描述顏色的方式：一個「深一點的紅」只需要調降 V 或 L，色相 H 保持不變。

對機器視覺而言，這個分離帶來一個關鍵好處：承 8.1.3 節，陽光下與陰影中的紅蘋果，雖然 RGB 差異巨大，但它們的 H(色相)卻相當接近——變的主要是 V(明度)。因此「以 H 為主要判準、對 V 放寬門檻」的顏色分割，遠比在 RGB 空間設範圍來得穩健。這正是 8.6 節顏色追蹤的核心策略。

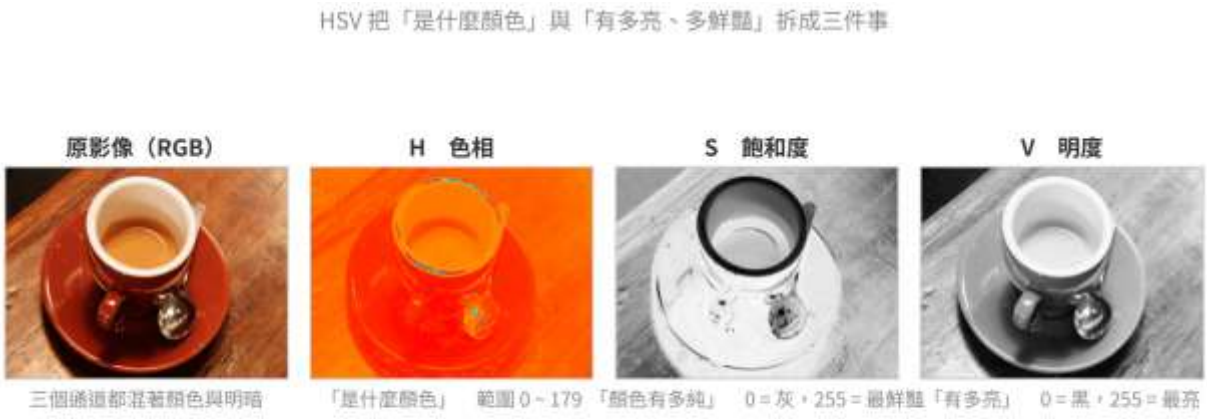


圖 8-5　把一張照片拆成 H、S、V 三個通道

8.2.1 幾何詮釋：圓柱、圓錐與雙圓錐

這兩個模型有三種常見的幾何視覺化方式，各自對應不同的理解角度：

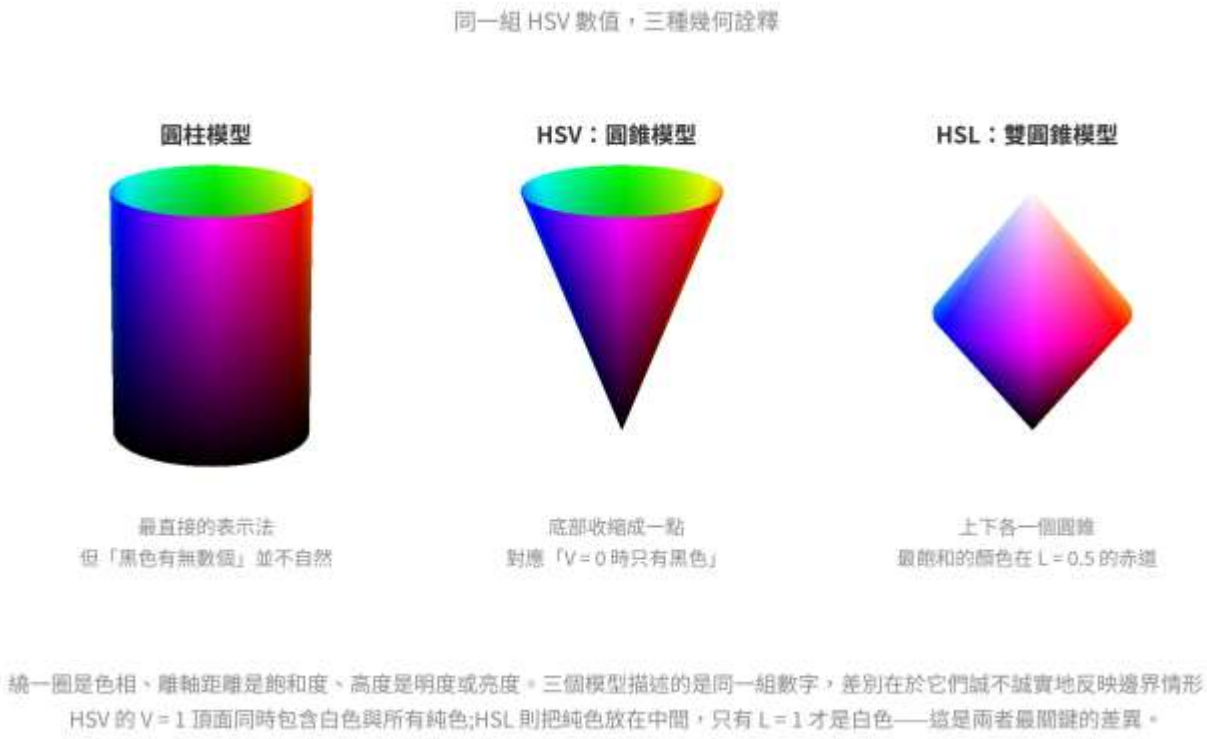


圖 8-6　圓柱、圓錐與雙圓錐：同一組數字的三種幾何說法

1. 圓柱模型：最直接的表示法。繞著圓周一圈是色相 H(0 度紅、120 度綠、240 度藍);離中心軸的半徑是飽和度 S(中心為灰，愈外圈愈鮮豔);沿著軸的高度是 V

或 L。缺點是它「浪費」了很多空間：當 V=0 時整層都是黑色，無論 H 與 S 為何，但圓柱仍然畫出一整個圓面。

2. 圓錐模型(HSV)：把圓柱底部收縮成一個點，正好對應「明度為零時只有黑色」的事實。頂面圓心為白色，頂面外緣是最飽和的純色。這是 HSV 最貼切的幾何。

3. 雙圓錐模型(HSL)：上下各一個圓錐，尖端分別是白(L=1)與黑(L=0)，最飽和的顏色出現在中間高度 L=0.5 的赤道圓周上。這反映了 HSL 的設計：亮度 L 是對稱的，從黑經過純色到白。

HSV 與 HSL 的關鍵差異就在這裡：HSV 的 V=1 頂面同時包含白色與所有純色(把白色和純紅都視為「最亮」);HSL 則把純色放在中間，只有 L=1 才是白色。

實務上的選擇原則：做顏色分割與追蹤時 HSV 較常用，因為「色相加飽和度」的門檻設定較直覺，且 V 通道直接對應「有多亮」便於放寬;而做顏色調整介面(如繪圖軟體的調色盤)則 HSL 較符合使用者對「亮度」的期待——把滑桿拉到最上面應該是白色，而非刺眼的純色。



圖 8-7　亮度掃過五倍範圍：哪個通道撐得住

8.2.2 OpenCV 的 HSV 數值範圍陷阱

這是初學者的第二大地雷(第一大是第 2 章的 BGR)。理論上色相 H 的範圍是 0 至 360 度，但 OpenCV 為了塞進 8 位元的 uint8(上限 255)，把 H 除以二，範圍變成 0 至 179;S 與 V 則是 0 至 255。因此網路上查到的「紅色 H 約為 0 或 360」，在 OpenCV 中要換算成 0 或 179。

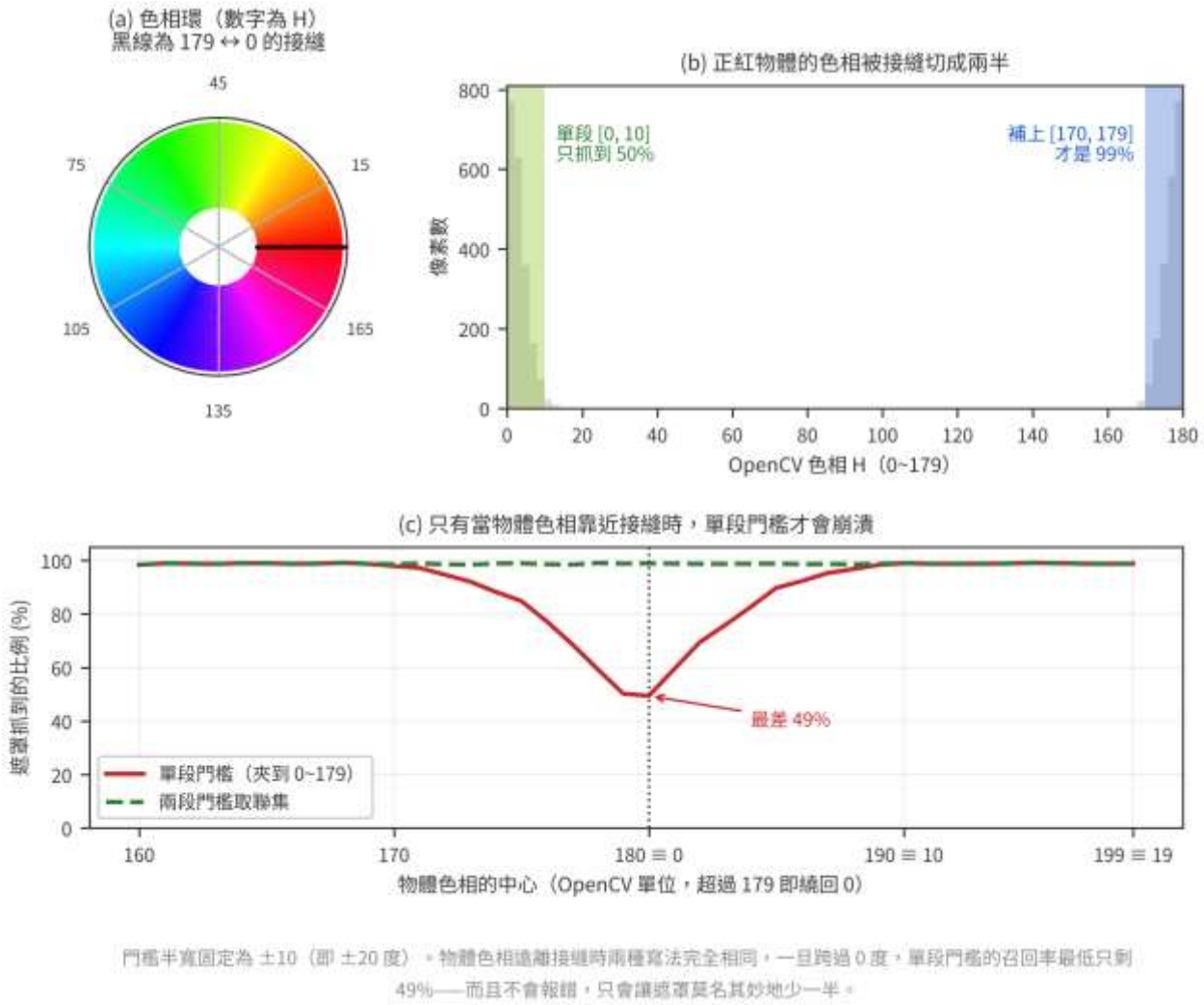
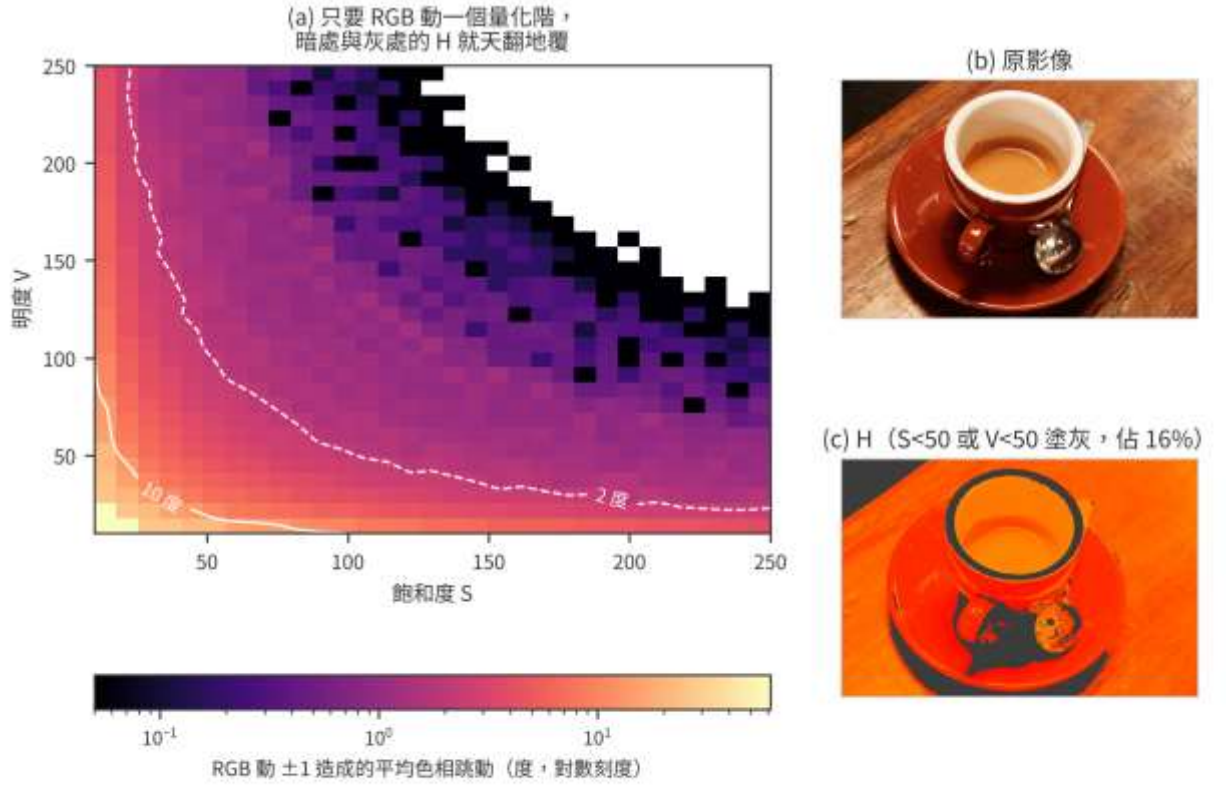


圖 8-8　OpenCV 的 H 只有 0 到 179，而紅色正好卡在接縫上

另一個必須注意的實務細節：紅色的色相跨越 0 度的接縫，因此偵測紅色時通常需要設定兩段區間再取聯集。這個「環狀資料的接縫問題」在處理角度類資料時普遍存在(第 12 章光流的方向、第 13 章 SIFT 的方向直方圖也都要處理)，值得記在心裡。

還有一個常被忽略的陷阱：當顏色很暗或很不飽和時(接近黑、白、灰),H 值變得極不穩定甚至無意義——想像圓錐的尖端，所有色相在那裡都收縮成同一點。因此顏

色分割時務必同時設定 S 與 V 的下限，把這些「色相無意義」的區域排除，否則會出現大量誤判。



實測同一張照片：對每個像素的 BGR 加上 −1~+1 的隨機擾動後，S<50 或 V<50 的區域色相平均跳動 2.4 度，其餘區域只跳動 0.4 度（相差 5.4 倍）；更極端的 S<25 或 V<25 則跳動 3.9 度。這就是顏色分割必須同時設定 S 與 V 下限的量化理由——否則演算法會把一堆「其實沒有顏色」的像素當成有效的色相來比對。

圖 8-9　暗處與灰處的色相：RGB 動一階就天翻地覆

程式 8-1 HSV 轉換與顏色門檻

```
import cv2, numpy as np

bgr = cv2.imread("robot.jpg")
hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)

# OpenCV:H 為 0~179、S 與 V 為 0~255
h, s, v = cv2.split(hsv)
print("H 範圍:", h.min(), h.max())

# --- 偵測藍色:單一區間即可 ---
#   注意 S 與 V 的下限:排除過暗與過灰的區域(H 在那裡無意義)
blue = cv2.inRange(hsv, (100, 120, 60), (130, 255, 255))

# --- 偵測紅色:跨越 0 度接縫,需兩段聯集 ---
red1 = cv2.inRange(hsv, (0, 120, 60), (10, 255, 255))
red2 = cv2.inRange(hsv, (170, 120, 60), (179, 255, 255))
red = cv2.bitwise_or(red1, red2)
```

```
# --- 驗證 8.1.3 節的說法:同一物體在明暗下的 RGB 與 HSV 變化 ---
bright = np.uint8([[[40, 40, 220]]])            # BGR 的亮紅
dark   = np.uint8([[[15, 15, 90]]])             # 同色但變暗
for name, px in [("亮", bright), ("暗", dark)]:
    hsv_px = cv2.cvtColor(px, cv2.COLOR_BGR2HSV)[0, 0]
    print(f"{name}: BGR={px[0,0]}, HSV={hsv_px}")
# 觀察:BGR 三值都大幅改變,但 H 幾乎不變 → 故以 H 為判準較穩健

cv2.imwrite("mask_red.png", red)
```

8.2.3 推導：RGB 轉 HSV 的公式是怎麼生出來的

幾乎每一本教科書都會列出 RGB 轉 HSV 的公式，而幾乎每一個學生看完都只記得「有一堆 if」。其實那些 if 不是硬湊出來的，它們是一個很單純的幾何操作留下的痕跡。只要把這件事想清楚，公式就不必背了。

先把 RGB 想成一個邊長為 1 的立方體，黑在原點 (0, 0, 0)，白在對角 (1, 1, 1)。連接黑與白的那條對角線上，三個分量永遠相等，所以那整條線就是灰階——這條線叫做中性軸。現在把這個立方體拿起來，讓中性軸對著你的視線方向，從白色那一端往黑色看過去。你看到的會是一個正六邊形：紅、黃、綠、青、藍、洋紅六個頂點依序排在圓周上，灰色落在正中央。HSV 的全部內容就是用極座標描述這張投影圖，再加上一個「離黑色多遠」的高度。

三個分量因此各有明確的幾何意義。明度 V = max(R, G, B)：這是立方體上「最接近白色的那一面」的座標，所以 V = 1 的那一層同時含有白色與所有最飽和的純色——這正是 8.2.1 節說 HSV 圓錐頂面既是白又是純色的原因。飽和度 S = (max − min) / max：分子 max − min 衡量三個分量彼此差多少，完全相等時為零（落在中性軸上，就是灰）;除以 max 是為了讓同一個色相在不同亮度下有相同的 S。色相 H 則是投影圖上的角度，而算角度的標準技巧就是先看「哪一個分量最大」，決定自己落在六邊形的哪一個 60 度扇區，再用另外兩個分量的差去算扇區內的偏移量。

於是那三個 if 的身分就清楚了：它們只是在問「現在人在哪一個扇區」。若 R 最大，H = 60 × ((G − B) / (max − min) mod 6);若 G 最大，H = 60 × ((B − R) / (max −

min) + 2);若 B 最大，H = 60 × ((R − G) / (max − min) + 4)。式子裡的 +2 與 +4 不過是「跳過前面兩個扇區」（每個扇區 120 度，除以 60 就是 2），而 mod 6 正是色相環繞回原點的那一刀——圖 8-8 的接縫問題，源頭就在這個 mod。

最後看分母。三條式子的分母都是 max − min，也就是 S 的分子。當一個顏色接近灰色時 max − min 趨近於零，分母趨近於零，H 的計算結果就會對輸入的微小變動極度敏感。這不是程式寫得不好，而是幾何上的必然：在六邊形的正中央，「角度」這個概念本來就沒有定義。圖 8-9 量到的那個「RGB 動一階、色相跳好幾十度」的現象，到這裡終於有了解釋——它就是一個除以接近零的數的典型症狀，和 7.1.5 節談的條件數是同一件事。

8.3 YCbCr 與 CIE Lab

8.3.1 YCbCr：為壓縮與傳輸而生

YCbCr 把亮度 Y 與兩個色差通道 Cb(藍色差)、Cr(紅色差)分開。它的設計動機非常務實：人眼對亮度細節的敏感度遠高於色彩細節，因此可以對 Cb、Cr 大幅降取樣(常見的 4:2:0 取樣就是把色度解析度砍成四分之一)，而人眼幾乎察覺不出差異。



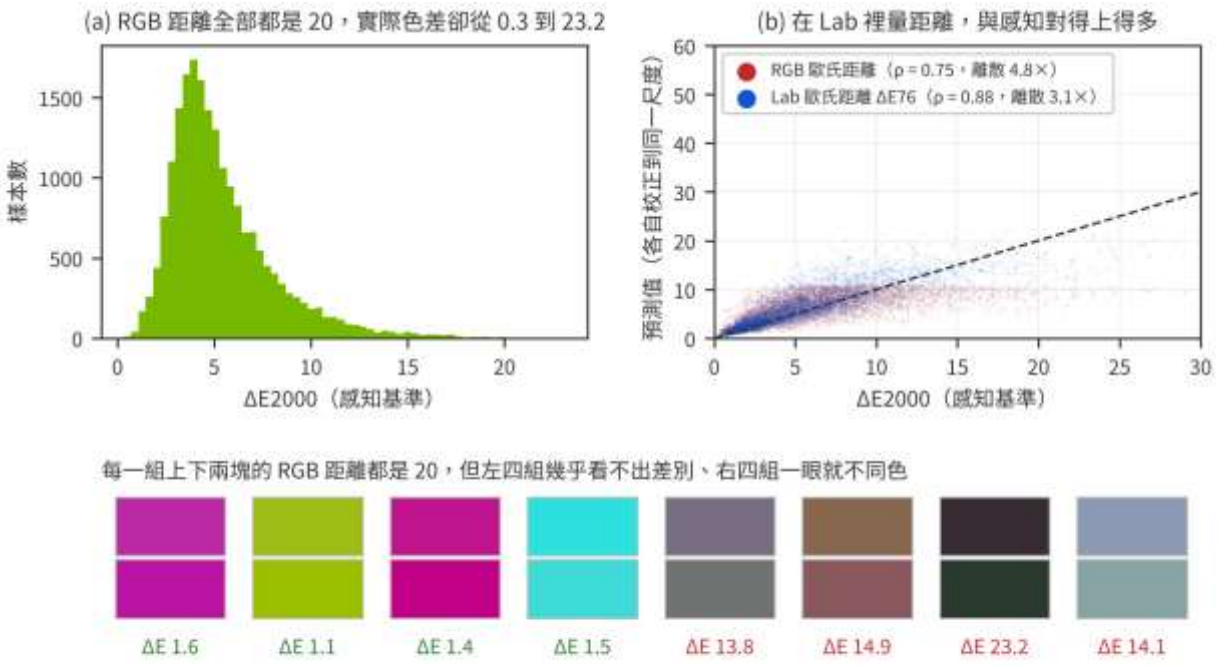
圖 8-10　為什麼砍色度不心疼，砍亮度會出事

這是 JPEG 壓縮與所有數位視訊標準的第一步——第 11 章談 JPEG 時會完整展開。它也是數位電視與類比電視相容的關鍵：黑白電視只讀 Y 通道即可正常顯示。

值得留意這個「把資源分配給人眼在意之處」的設計思想，它在本書中反覆出現：第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、本節的色度降取樣、第 11 章 JPEG 對高頻係數用較大的量化步長。三者都是同一種智慧——不是所有資訊都同等重要，把位元花在刀口上。

8.3.2 CIE Lab：為量測色差而生

CIE Lab 由國際照明委員會制定，目標是感知均勻(Perceptually Uniform)：在這個空間中，兩個顏色的歐氏距離大致正比於人眼感受到的色差大小。



以 CIE 現行標準 ΔE2000 為感知基準（本身仍是模型而非真理），比較兩種「算距離」的方式：Spearman 等級相關由 0.75 提高到 0.88，預測比值的離散度由 4.8 倍降到 3.1 倍。這就是「感知均勻」的具體含意——不是 Lab 完美，而是在 Lab 裡量出來的距離，比在 RGB 裡量的更接近人眼的感受。

圖 8-11　感知均勻的意思：同樣的 RGB 距離，色差可以差 70 倍

這在 RGB 中是做不到的——RGB 空間裡同樣的數值差距，在不同區域造成的視覺差異可能天差地別(人眼對綠色的細微變化極為敏感，對藍色則相對遲鈍)。Lab 的 L 為亮度(0 黑到 100 白),a 為綠紅軸，b 為藍黃軸。

應用場合：印刷與紡織的品管(以 ΔE 色差值判定是否合格，通常 ΔE 小於 1 表示人眼幾乎無法分辨)、色彩校正、以及第 5 章介紹過的技巧——對彩色影像做 CLAHE 時，轉到 Lab 只處理 L 通道，色彩才不會失真。

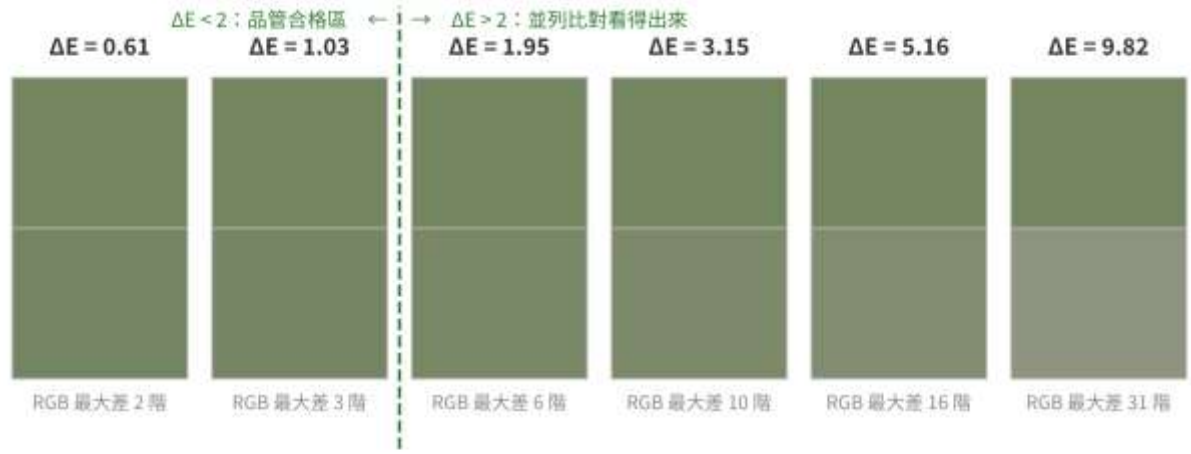


圖 8-12　ΔE 要多大才看得出來：一條親自檢驗的色差尺

色彩空間	設計目的	通道意義	典型應用
RGB	對應感光元件與螢幕	紅、綠、藍強度	顯示、儲存
HSV	符合人類對顏色的描述	色相、飽和度、明度	顏色分割與追蹤
HSL	亮度對稱、便於調色	色相、飽和度、亮度	調色盤、繪圖軟體
YCbCr	利用人眼對色度不敏感以壓縮	亮度、藍色差、紅色差	JPEG、視訊編碼
CIE Lab	感知均勻，可量測色差	亮度、綠紅軸、藍黃軸	品管色差、色彩校正

表 8-1 常用色彩空間比較

8.3.3 推導：0.299、0.587、0.114 這三個數字從哪裡來

把彩色轉成灰階時，幾乎所有的程式庫都在算 Y = 0.299R + 0.587G + 0.114B。這三個係數看起來很隨意，加起來剛好是 1，而且綠色的權重是藍色的五倍以上。它們既不是誰拍腦袋決定的，也不是實驗湊出來的——它們是從一組顏色座標解一個線性方程式解出來的。

出發點是這樣一個要求：我們想要一個亮度通道，使得「白色的亮度等於 1」，而且這個亮度盡可能對應人眼感受到的明暗。CIE 1931 系統裡的 Y 分量正是為此設計

的（它的定義就是用人眼的明視覺光譜效率曲線加權積分而來）。所以問題變成：已知螢幕三個原色各自的 CIE 座標，請問要用多少比例混合，才能得到 Y？

動手算。以類比電視時代的 NTSC 原色為例，三個原色的色度座標分別是紅 (0.67, 0.33)、綠 (0.21, 0.71)、藍 (0.14, 0.08)，而參考白點為 (0.310, 0.316)。色度座標的定義是 x = X/(X+Y+Z)、y = Y/(X+Y+Z)，因此每個原色的 XYZ 可以寫成 (x/y, 1, (1−x−y)/y) 乘上一個待定的縮放係數。把三個待定係數排成一個向量，要求「三個原色全開時總和等於白點的 XYZ」，就得到一個三元一次方程組。解出來之後，每個原色的 Y 分量就是我們要的權重，而答案是 0.2990、0.5864、0.1146——四捨五入到三位小數，就是課本上那三個數字。總和為 1 不是巧合，而是「白色的 Y 等於 1」這個條件直接要求的結果。想親手驗算的讀者，這個三元一次方程組用 numpy 的 linalg.solve 三行就解得出來，把原色座標換成別的標準也立刻得到那個標準的係數。

這個推導帶出兩個常被忽略的實務重點。第一，係數會隨原色而變。現代的高畫質電視採用 Rec. 709 原色，同樣的算法解出來是 0.2126、0.7152、0.0722;超高畫質的 Rec. 2020 又是另外一組 0.2627、0.6780、0.0593。OpenCV 的 cvtColor 預設仍用 0.299 那一組，所以若你的流程混用了不同標準，灰階值會有百分之幾的差異——這在一般應用無傷大雅，在色彩量測或影像比對上卻可能造成系統性偏差。

第二，綠色權重特別大，是因為人眼的錐狀細胞在中波段最敏感（圖 8-1），而不是因為綠色比較重要。這一點把本書的三條線索串了起來：第 3 章的拜爾陣列讓綠色佔一半、本節的亮度係數讓綠色佔六成、8.3.1 節的色度降取樣把資源留給 Y——三個看似無關的設計，背後是同一句話：把位元花在人眼在意的地方。

8.4 虛擬色彩：讓看不見的資訊現形

虛擬色彩(Pseudocolor，又稱偽彩色)是把灰階或單一數值的資料，依照一張色彩對照表(Colormap)映射成彩色影像。

它的目的不是「還原真實顏色」——這些資料本來就沒有顏色——而是利用人眼分辨顏色的能力遠強於分辨灰階的特性：人類大約只能分辨數十階灰度，卻能分辨數萬種顏色。把資料的細微差異映射到色彩，就能讓原本看不出來的結構一目了然。

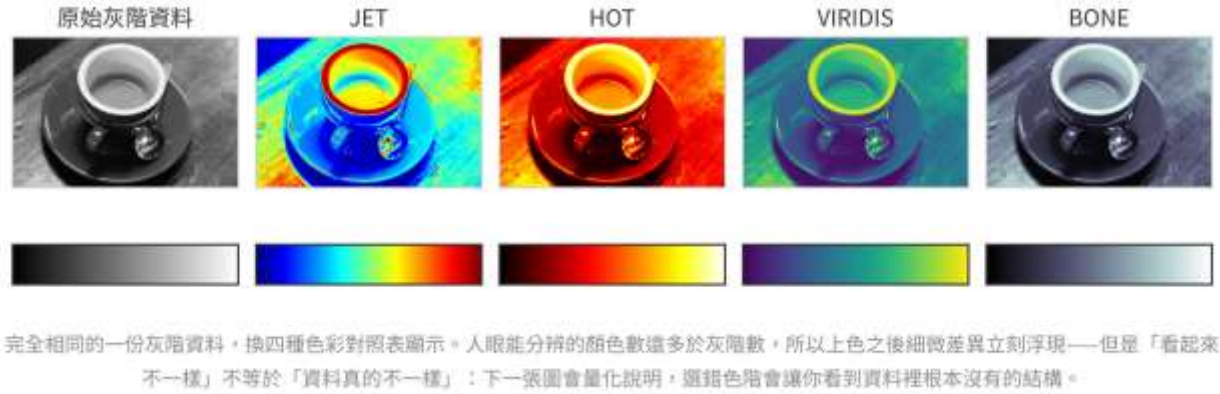


圖 8-13　同一份灰階資料，四種色階

應用場景極廣：熱影像儀把溫度映射成藍到紅的色階;醫學影像用色彩標示血流速度或代謝強度;衛星遙測把近紅外波段染成紅色以突顯植被;深度相機(第 17 章)把距離映射成彩虹色;第 12 章的稠密光流用色相表示運動方向;第 15、16 章的神經網路熱力圖也是靠虛擬色彩顯示「AI 在看哪裡」。

8.4.1 色階選擇的專業考量

經典的 JET(彩虹色)雖然搶眼，但在科學視覺化領域近年頗受批評，原因有三：其一，它的亮度變化不是單調遞增的(中間的黃綠色比兩端更亮)，容易在資料中製造出實際不存在的「邊界」，誤導判讀;其二，對色盲使用者不友善(約百分之八的男性有紅綠色盲);其三，轉成黑白列印後資訊完全喪失。

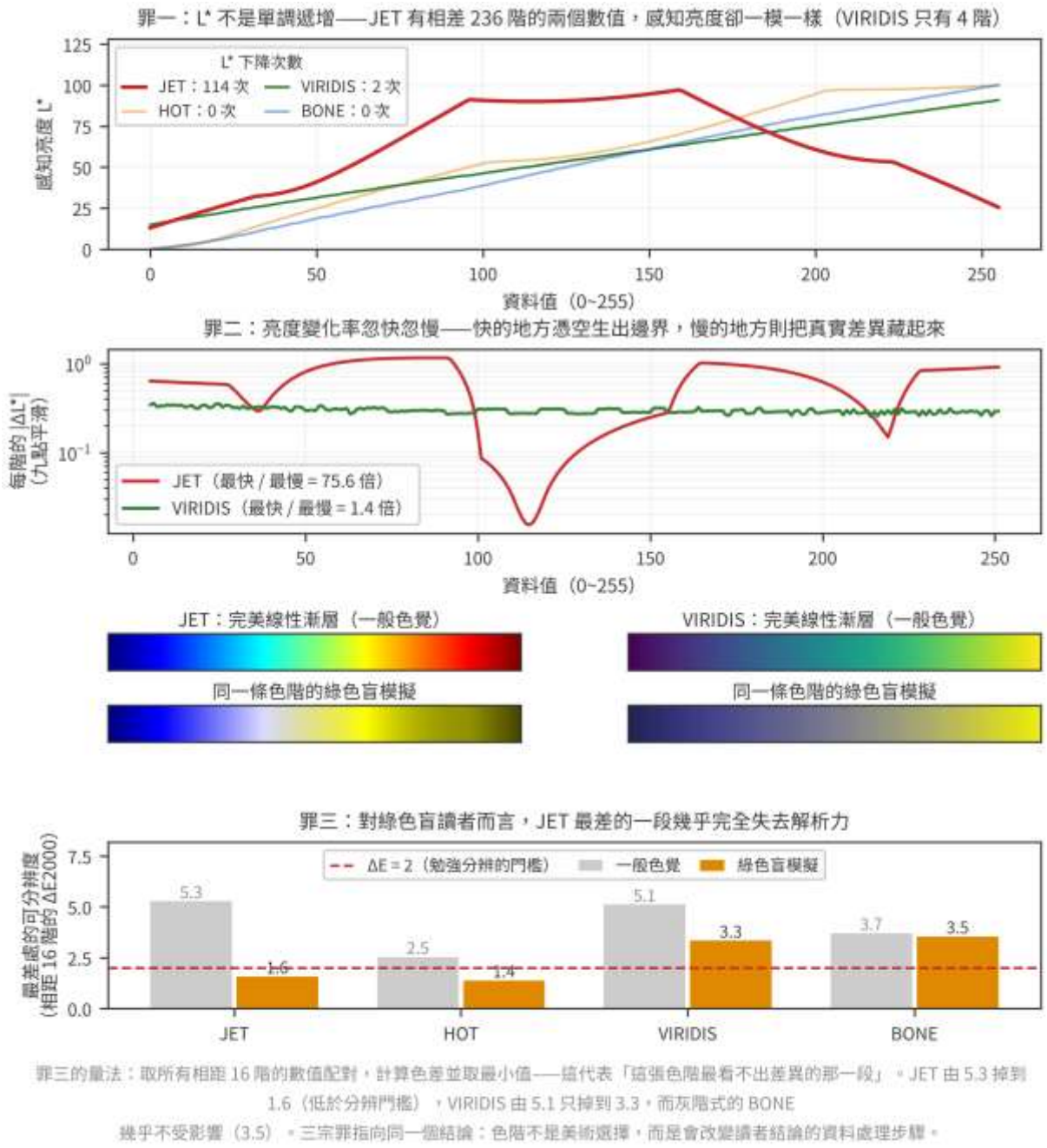


圖 8-14　JET 的三宗罪，每一宗都量得出來

VIRIDIS、MAGMA 等感知均勻的色階已成為新的推薦選項——它們的亮度單調遞增、色盲友善、且黑白列印後仍保有層次。在論文與報告中選擇 colormap，也是一種專業素養;這個考量與 8.3.2 節 CIE Lab 的「感知均勻」是同一種思想在視覺化上的應用。

程式 8-2 虛擬色彩與色階的檢驗

```
import cv2
```

```
gray = cv2.imread("thermal_raw.png", cv2.IMREAD_GRAYSCALE)

# 常用色彩對照表
jet     = cv2.applyColorMap(gray, cv2.COLORMAP_JET)      # 經典彩虹(慎用)
hot     = cv2.applyColorMap(gray, cv2.COLORMAP_HOT)      # 熱影像風格
viridis = cv2.applyColorMap(gray, cv2.COLORMAP_VIRIDIS)  # 感知均勻,推薦
bone    = cv2.applyColorMap(gray, cv2.COLORMAP_BONE)     # 醫學影像風格

# 檢驗色階是否「亮度單調遞增」:轉回灰階看是否仍為漸層
import numpy as np
ramp = np.tile(np.arange(256, dtype=np.uint8), (50, 1))
for name, cm in [("jet", cv2.COLORMAP_JET),
                 ("viridis", cv2.COLORMAP_VIRIDIS)]:
    colored = cv2.applyColorMap(ramp, cm)
    back = cv2.cvtColor(colored, cv2.COLOR_BGR2GRAY)
    diff = np.diff(back[0].astype(np.int16))
    print(f"{name}: 亮度單調遞增? {np.all(diff >= 0)}")
    # JET 會出現 False → 這正是它造成假邊界的原因

cv2.imwrite("pseudo_viridis.png", viridis)
```

8.5 白平衡與色彩校正

回到叫獸開講的核心問題：如何讓機器扣掉光源的影響?這正是白平衡(White Balance)要解決的事，也是人腦色彩恆常性的工程版本。

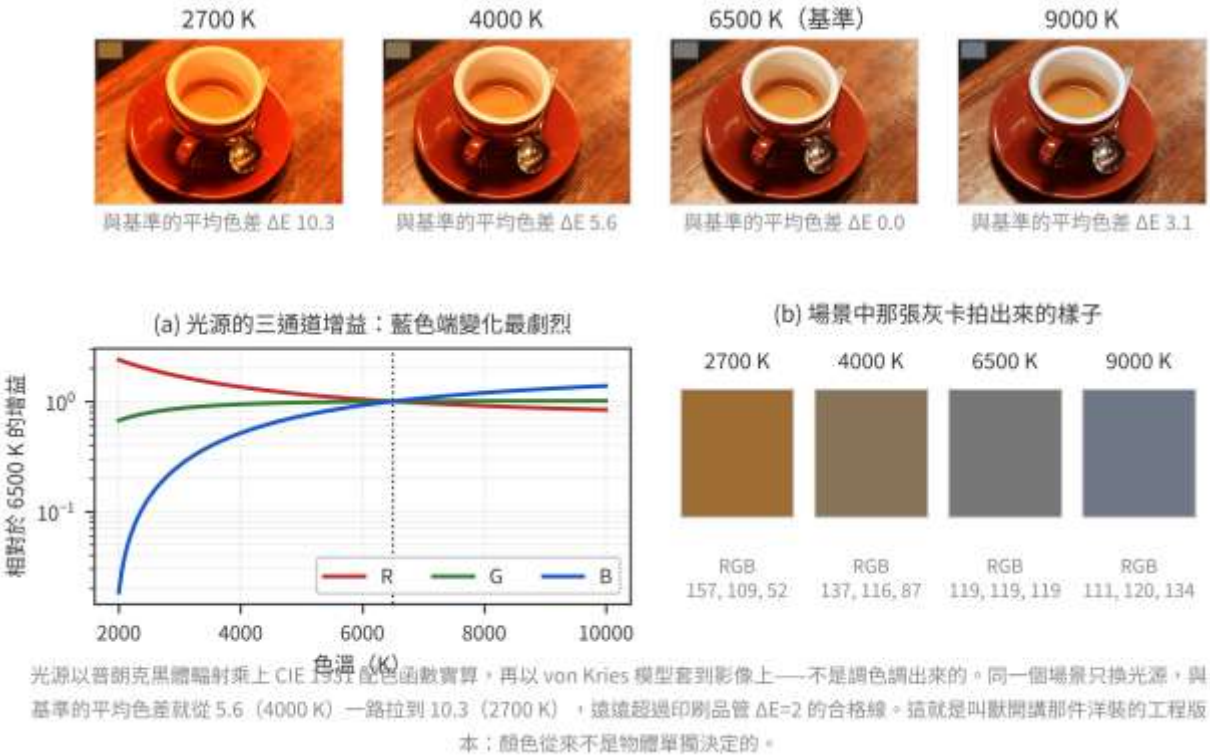
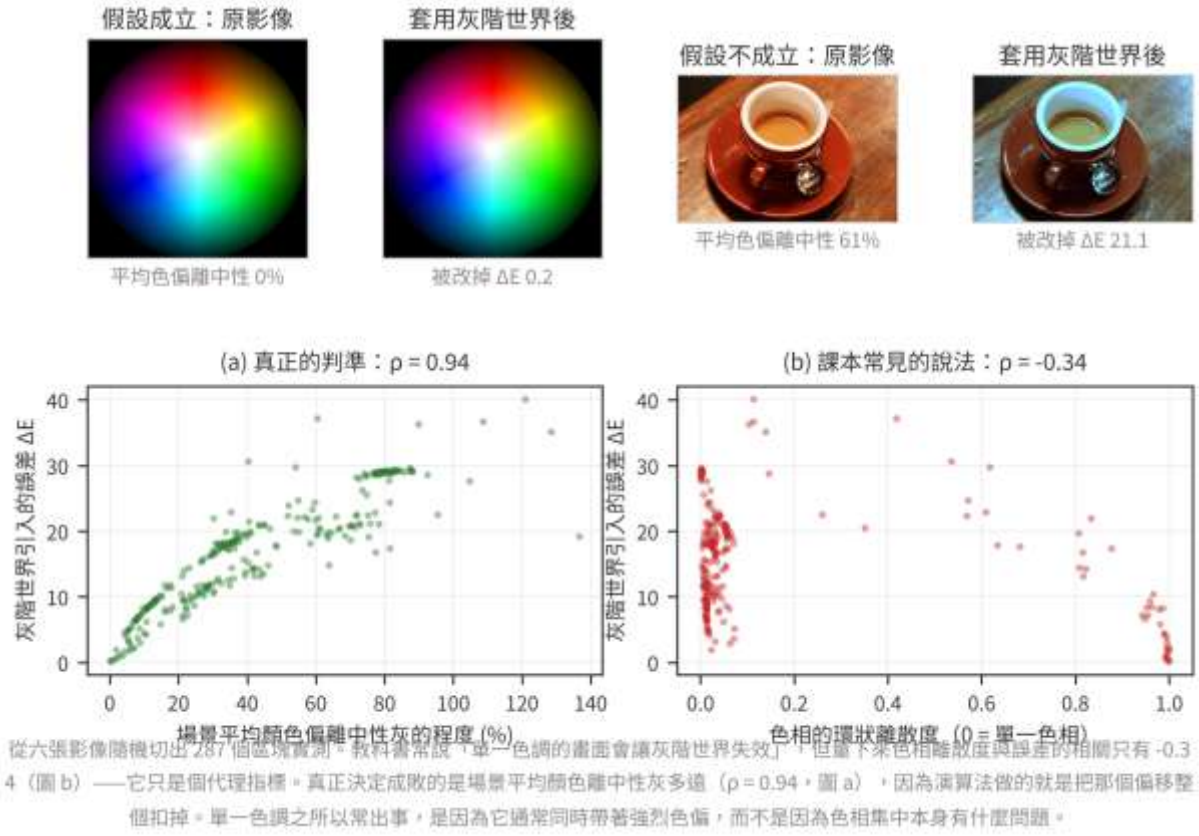


圖 8-15　只換光源，同一個場景就換了顏色

8.5.1 灰階世界假設

最簡單的演算法是灰階世界假設(Gray World)：假設整個場景的平均顏色應該是中性灰。若實測的平均偏藍，代表光源偏藍，於是把藍通道整體調降、紅通道調升。

它在色彩豐富的場景中效果不錯，但遇到「整片草地」、「整面紅牆」、「大片藍天」這類單一色調的畫面就會失效——因為前提假設不成立，演算法會誤以為光源偏綠(或偏紅、偏藍)，硬是把草地校正成灰色。這是一個很好的提醒：所有演算法都建立在假設之上，而工程師的責任是判斷假設是否成立。



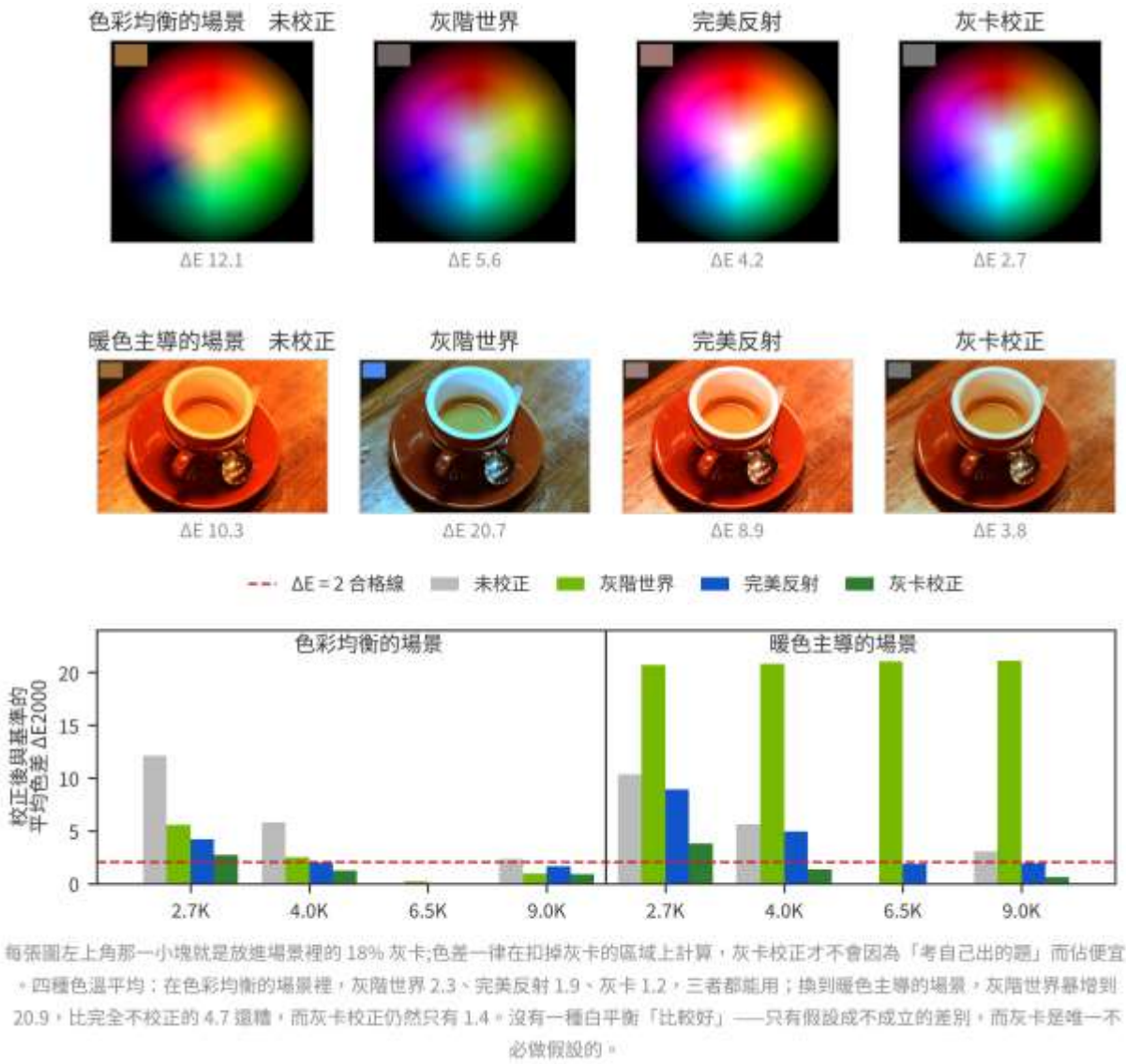
從六張影像隨機切出 287 個區塊實測。我科書常說「單一色調的畫面會讓灰階世界失效」，但量下來色相離散度與誤差的相關只有 -0.34（圖 b）——它只是個代理指標。真正決定成敗的是場景平均顏色離中性灰多遠（ρ = 0.94，圖 a），因為演算法做的就是把那個偏移整個扣掉。單一色調之所以常出事，是因為它通常同時帶著強烈色偏，而不是因為色相集中本身有什麼問題。

圖 8-16　灰階世界什麼時候會失效：真正的判準不是色相

8.5.2 完美反射假設與色卡校正

第二種是完美反射假設(White Patch / Retinex)：找出影像中最亮的像素，假定它應該是純白，據此計算各通道的增益。它對「畫面中確實有白色物體」的場景很準，但遇到過曝或高光反射就會誤判——因為過曝的像素已經飽和，其色彩資訊已經喪失。

工業場合最可靠的做法則是第三種：在畫面中放置標準色卡(如 24 色的 ColorChecker)，以已知的參考色計算完整的色彩校正矩陣。這不只是白平衡，而是把整個色彩響應校正到標準——這也是叫獸開講思考題的標準答案之一，更是農產品分級、印刷打樣、醫學影像等對顏色準確度有要求的場合的標準流程。



每張圖左上角那一小塊就是放進場景裡的 18% 灰卡;色差一律在扣掉灰卡的區域上計算，灰卡校正才不會因為「考自己出的題」而佔便宜。四種色溫平均：在色彩均衡的場景裡，灰階世界 2.3、完美反射 1.9、灰卡 1.2，三者都能用；換到暖色主導的場景，灰階世界暴增到 20.9，比完全不校正的 4.7 還糟，而灰卡校正仍然只有 1.4。沒有一種白平衡「比較好」——只有假設成不成立的差別，而灰卡是唯一不必做假設的。

圖 8-17　三種白平衡 × 兩種場景 × 四種色溫的完整成績

至於現代的第四種路線：第 5 章 Night Sight 的學習式白平衡——用神經網路從大量標註資料中學習「什麼樣的影像看起來白平衡正確」。它在傳統方法失效的困難場景(如夜景、單色調畫面)表現優異，但也繼承了深度學習的侷限(第 15 章)：遇到訓練分布外的場景可能失準，且難以解釋。

8.5.3 推導：灰階世界為什麼會失效，以及失效多少

8.5.1 節說灰階世界在單一色調的畫面中會失效。這句話對，但不夠精確——圖 8-16 實測了 287 個真實影像區塊，發現色相的集中程度與誤差的相關係數只有 −0.34，而「場景平均顏色離中性灰多遠」的相關係數高達 0.94。這一小節把這件事算清楚。

先把演算法寫成數學式。設場景中每個像素的反射率為 $\rho(x)$，光源在三個通道的強度為 L，則相機拍到的值為 $I(x) = L \odot \rho(x)$（$\odot$ 表示逐通道相乘）。對整張影像取平均，得到 $\text{mean}(I) = L \odot \text{mean}(\rho)$。灰階世界做的事，是把每個通道除以該通道的平均值，也就是輸出 $I(x) / \text{mean}(I) = \rho(x) / \text{mean}(\rho)$。請注意這個結果：$L$ 被完整地消掉了。這代表灰階世界其實是一個「無論光源是什麼都一定成立」的恆等式——它永遠會把光源除乾淨。

那問題出在哪裡？出在最後那個 $\text{mean}(\rho)$。我們要的答案是 $\rho(x)$ 本身，拿到的卻是 $\rho(x) / \text{mean}(\rho)$。只有當場景的平均反射率在三個通道上相等（也就是 $\text{mean}(\rho)$ 正比於 $(1, 1, 1)$，場景平均真的是中性灰）時，這個除法才只是整體縮放，不改變顏色。否則殘留的誤差就是 $\text{mean}(\rho)$ 本身的色偏，與光源完全無關。這解釋了圖 8-17 裡一個乍看詭異的結果：在暖色主導的場景中，灰階世界在四種色溫下的誤差都是 21 左右，幾乎不隨光源變化——因為那個誤差根本不是光源造成的，而是場景自己的色偏被當成光源扣掉了。

誤差的大小也因此可以預估。殘留的通道增益是 $\text{mean}(\rho)$ 各通道與其平均的比值，所以「場景平均顏色偏離中性灰的百分比」幾乎就是誤差的直接度量——這正是圖 8-16(a) 那條相關係數 0.94 的曲線。單一色調的場景之所以常出事，是因為它通常同時帶著強烈色偏;但一張色相集中卻剛好接近中性的照片（例如灰階的雪景加一點淡藍）並不會讓灰階世界失效，而一張色相很分散卻整體偏暖的照片（例如黃昏的市集）反而會。把判準從「色相散不散」改成「平均離不離中性」，你才真的知道什麼時候可以用它。

最後是工程上的收尾。既然誤差來自 mean(ρ) 的未知色偏，那麼補救的方向只有兩個：一是讓 mean(ρ) 變成已知——在畫面中放一張反射率已知的灰卡，用它來算增益，這就是 8.5.2 節的色卡校正，也是圖 8-17 中唯一在兩種場景都穩定的方法;二是乾脆不讓 L 變動——鎖住光源，問題就不存在。這兩條路都呼應 7.6.3 節的層次原則：能在源頭解決的，不要留到演算法去補。

程式 8-3 兩種白平衡演算法與其失效情境

```
import cv2, numpy as np

def gray_world(img):
    """灰階世界白平衡:假設場景平均為中性灰"""
    result = img.astype(np.float32)
    avg_b, avg_g, avg_r = (result[:,:,0].mean(),
                           result[:,:,1].mean(),
                           result[:,:,2].mean())
    avg_gray = (avg_b + avg_g + avg_r) / 3
    result[:,:,0] *= avg_gray / avg_b
    result[:,:,1] *= avg_gray / avg_g
    result[:,:,2] *= avg_gray / avg_r
    return np.clip(result, 0, 255).astype(np.uint8)

def white_patch(img, percentile=99):
    """完美反射假設:以最亮的像素作為白色參考"""
    result = img.astype(np.float32)
    for c in range(3):
        ref = np.percentile(result[:,:,c], percentile)  # 取分位數較穩健
        result[:,:,c] *= 255.0 / max(ref, 1)
    return np.clip(result, 0, 255).astype(np.uint8)

img = cv2.imread("yellowish_indoor.jpg")
cv2.imwrite("wb_grayworld.jpg", gray_world(img))
cv2.imwrite("wb_whitepatch.jpg", white_patch(img))

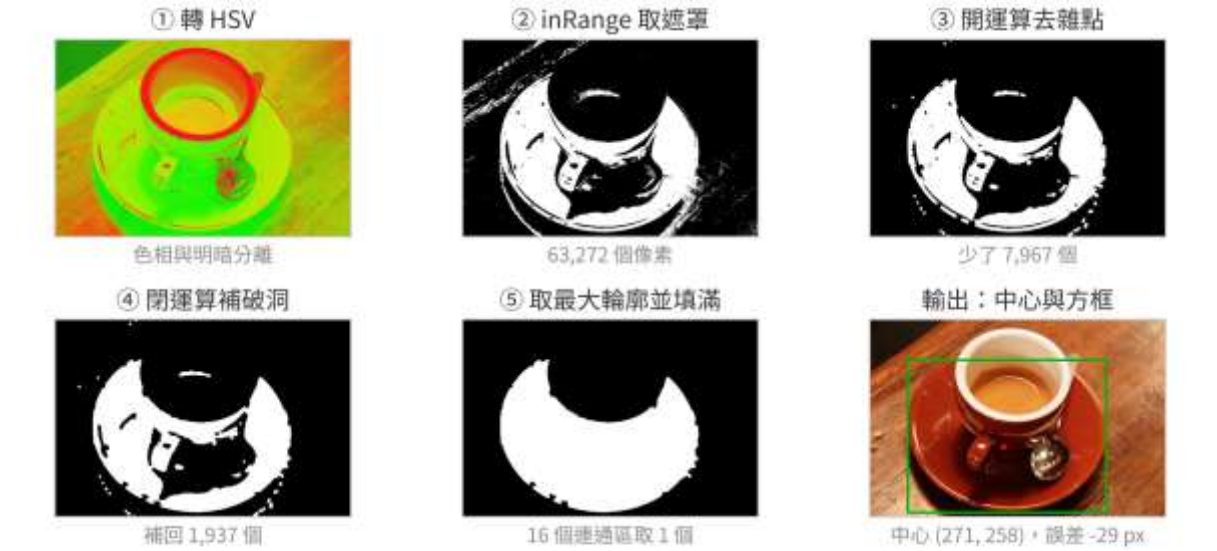
# 驗證失效情境:對一張「整片草地」的影像套用灰階世界
grass = cv2.imread("grass_field.jpg")
cv2.imwrite("wb_grass_fail.jpg", gray_world(grass))
# 觀察:草地被硬生生校正成灰綠色 → 假設不成立時演算法失效
```

8.6 顏色分割與物件追蹤

把前面所學組合起來，就得到機器人視覺最基礎也最實用的技能：用顏色找出目標物並持續追蹤。

標準流程五步驟：轉 HSV(利用色相對光照變化較穩健)→ 以 inRange 產生二值遮

罩 → 用型態學運算(第 10 章)清除雜點與補洞 → 找輪廓取最大者(第 9 章)→ 計算中心座標並輸出控制訊號。這正是循跡機器人、色球追蹤、簡易分揀系統的骨幹。



每一步的像素數都是實跑出來的。目標是那只紅色托盤。它的色相在 2~6 之間，而桌面木紋是 10~15，兩者很近——真正把它們分開的是飽和度（托盤 230 以上、木紋不到 200）。紅色跨越接縫，所以第 ② 步用了 H 0~8 與 H 172~179 兩段取聯集；S 與 V 的下限設在 195 與 50，把暗處與灰處那些色相無意義的像素擋在門外（圖 8-9）。第 ⑤ 步從 16 個連通區挑出最大的一個並填滿外輪廓，像素數反而變多（57,242 → 76,324）——因為托盤是個環，填滿外輪廓等於把中間那只杯子一起算進來。要抓取整組杯盤，這樣是對的；要量托盤本身的面積，這樣就錯了。填不填，取決於下游任務，不是取決於哪個比較漂亮。

圖 8-18　顏色分割五步驟，每一步的像素數都在上面

參數設定的實務心法：對 H 設窄門檻(因為色相是主要判準)、對 S 與 V 設寬門檻但要有下限(容許明暗變化，同時排除色相無意義的暗處與灰處)。這個「窄 H、寬 V」的原則，直接源自 8.1.3 與 8.2 節對 RGB 與 HSV 的比較。

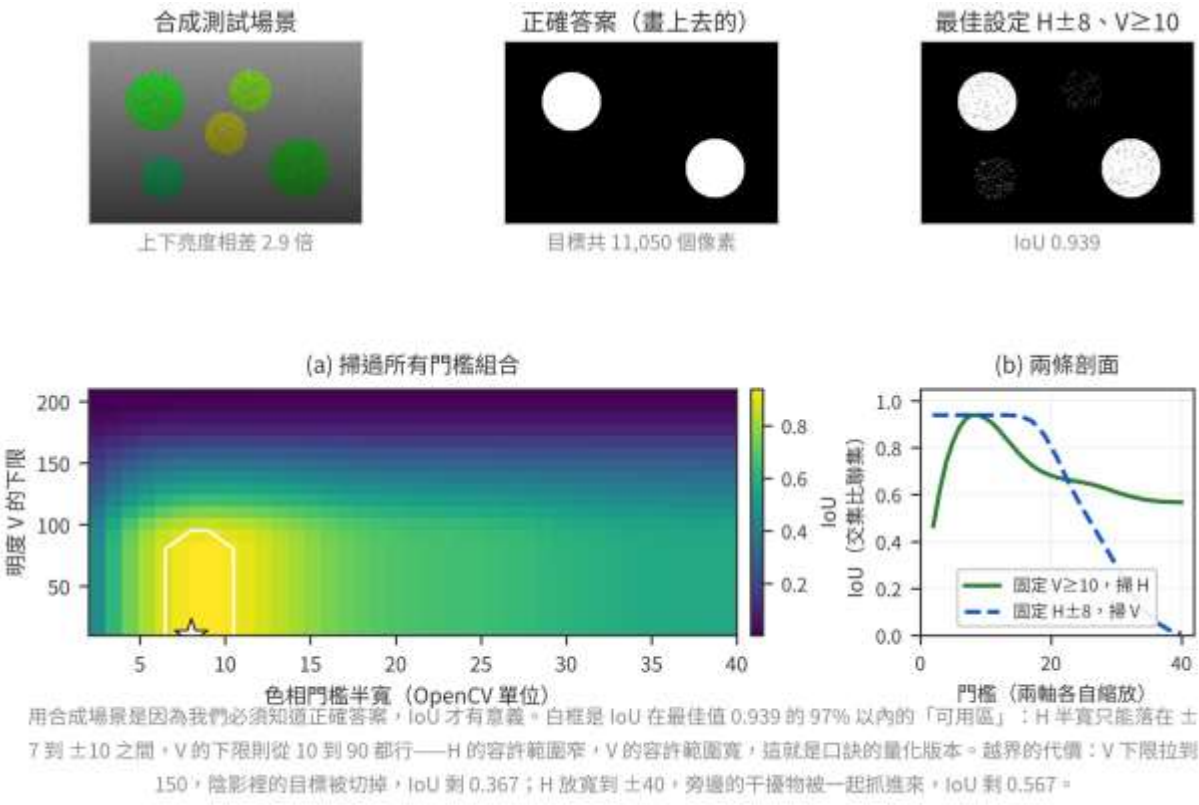


圖 8-19　「窄 H、寬 V」的量化版本：掃過所有門檻組合

程式 8-4 HSV 顏色追蹤與機器人控制訊號

```
import cv2, numpy as np

cap = cv2.VideoCapture(0)          # 或 ESP32-CAM 的串流網址(第 2 章)
kernel = np.ones((5, 5), np.uint8)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    frame = gray_world(frame)                  # 白平衡前處理(8.5 節)
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # 窄 H、寬 V,但 S 與 V 有下限
    mask = cv2.inRange(hsv, (35, 100, 60), (85, 255, 255))   # 綠色物件

    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)    # 去雜點
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)   # 補破洞

    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
    if cnts:
        c = max(cnts, key=cv2.contourArea)
        if cv2.contourArea(c) > 500:
            x, y, w, h = cv2.boundingRect(c)
            cx, cy = x + w // 2, y + h // 2
```

```
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)

            # 機器人控制訊號:依偏差量決定轉向
            err = cx - frame.shape[1] // 2
            cmd = "左轉" if err < -30 else ("右轉" if err > 30 else "直行")
            cv2.putText(frame, f"{cmd} (err={err})", (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    cv2.imshow("tracking", frame)
    if cv2.waitKey(1) == 27:
        break

cap.release(); cv2.destroyAllWindows()
```

8.7 應用案例

8.7.1 案例一：農產品的成熟度分級

情境：番茄、香蕉、水果的成熟度可由顏色判斷，是雲林在地農業自動化最直接的應用。做法：以 HSV 或 Lab 空間萃取顏色特徵(如色相的平均值與分布)，建立成熟度與顏色的對應關係，再以第 14 章的分類器或簡單的門檻規則分級。

這個案例最能凸顯叫獸開講的教訓：光源是成敗的關鍵。同一顆番茄在陰天與晴天、上午與傍晚拍攝，顏色可能落到不同的等級。因此實務上的必要措施是——建立封閉的檢測箱、使用穩定的 LED 光源、並在每次拍攝時納入標準色卡校正。若做不到這些(如田間直接拍攝)，就必須用第 15 章的資料增強讓模型學會適應光線變化，或改用對光源較不敏感的特徵。

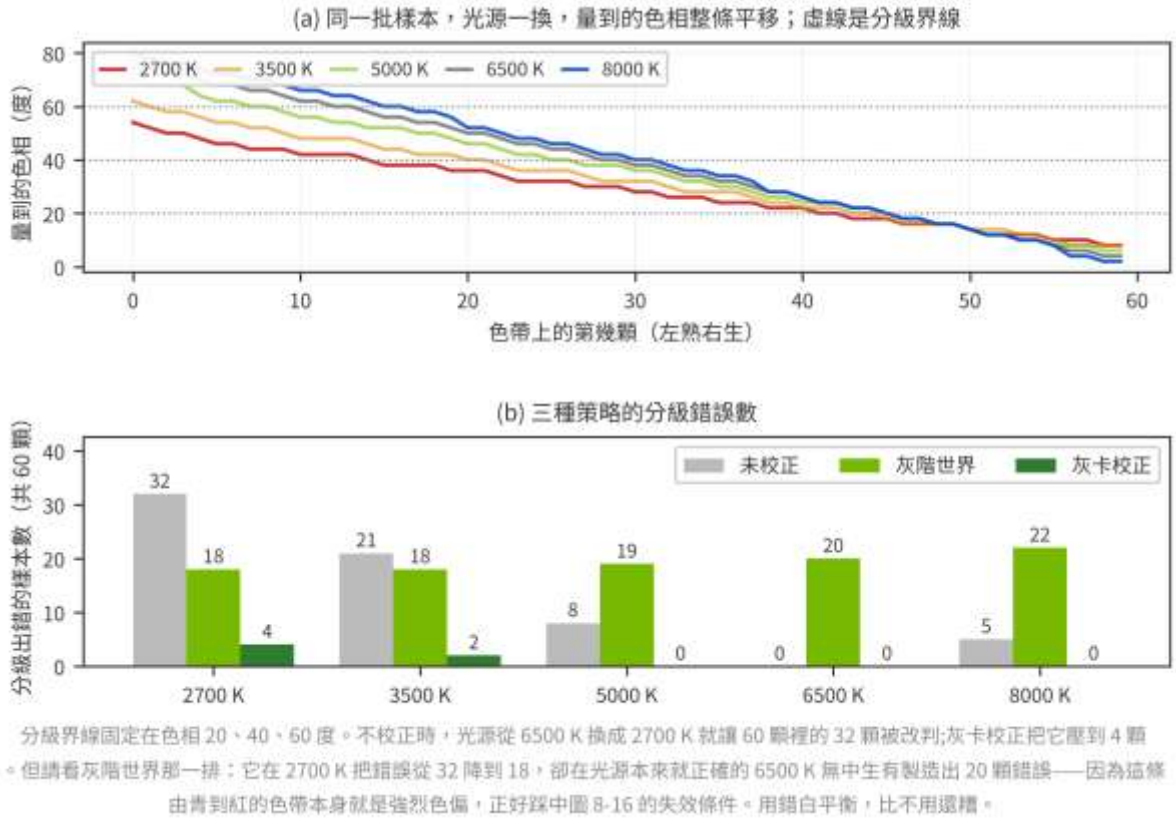


圖 8-20　成熟度分級：光源一換，六十顆裡三十二顆被改判

8.7.2 案例二：醫學與科學的虛擬色彩

情境：熱影像儀顯示體溫分布、超音波都卜勒顯示血流方向與速度、功能性磁振造影顯示腦部活動強度。這些資料本身都是單一數值(溫度、速度、訊號強度)，沒有顏色，必須透過虛擬色彩才能讓人判讀。

這裡有一個攸關判讀正確性的專業考量，呼應 8.4.1 節：色階的選擇會直接影響醫師「看到什麼」。用 JET 彩虹色階顯示的溫度分布，可能因為亮度非單調而讓人誤以為某處有明顯的邊界或異常;而選用感知均勻的色階，則能讓資料的真實漸變忠實呈現。此外，色階的數值範圍(對應第 5.7.2 節的視窗化)同樣關鍵——同一份資料用不同的範圍映射，可以讓病灶明顯或完全隱形。這再次呼應第 7、16 章的倫理主題：視覺化的選擇不是中性的。

8.7.3 案例三：工業 AOI 的光源設計

情境：檢測電子零件的顏色標記、判斷線材的色碼、辨識產品標籤。這類任務在

受控環境下極為可靠，關鍵完全在於「控制變因」。

工程上的標準做法包括：使用高演色性、色溫穩定的 LED 光源(廉價 LED 的光譜不連續，會使某些顏色偏移);設計光源的方向與擴散方式以避免高光反射(反光處會過曝，色彩資訊喪失);以遮光罩隔絕環境光的干擾;定期以標準色卡驗證色彩響應是否漂移(LED 隨使用時間會衰減與偏色)。

這個案例是叫獸開講思考題的完整答案：與其設計一個「能對抗任何光源」的聰明演算法，不如從源頭消除光源的變異——這在可控的產線環境中，永遠是更務實、更可靠的工程選擇。這也呼應第 7.6.3 節「三個層次的解法應以光學機構層為優先」的原則。

8.7.4 案例四：色彩在深度學習中的角色

情境：訓練第 15、16 章的模型時，色彩相關的決策比想像中更多。

第一，輸入格式：大多數預訓練模型期待 RGB 順序，而 OpenCV 讀進來是 BGR(第 2 章)——這個順序錯誤是常見的 bug，會使準確率莫名下降。第二，資料增強：隨機調整色相、飽和度、亮度(第 16 章 YOLO 的 hsv_h、hsv_s、hsv_v 參數)是標準做法，目的正是讓模型對光源變化不敏感——這是「用學習取代白平衡」的思路。第三，是否轉灰階：若任務與顏色無關(如形狀辨識)，轉灰階能減少三分之二的計算量;但若顏色是關鍵線索(如成熟度、警示標誌)，則絕不能丟棄。

這些決策說明了一件事：即使進入深度學習時代，對色彩的理解仍然是工程師必備的判斷力來源。模型不會替你決定該用哪個色彩空間、該不該做白平衡——那是你的工作。

8.8 本章與全書的觀念連結總表

本章觀念	相關章節	如何延伸

BGR 順序	第 2、15、16 章	OpenCV 的歷史包袱;深度模型期待 RGB
拜爾陣列與去馬賽克	第 3、4 章	每像素僅記錄一色，需內插補齊
只處理亮度通道	第 5 章	彩色影像做 CLAHE 應轉 Lab 只等化 L
色度降取樣	第 11 章	JPEG 第一步的 YCbCr 與 4:2:0
虛擬色彩	第 10、12、15 章	距離轉換視覺化;稠密光流;神經網路熱力圖
白平衡與色彩恆常性	第 5、15 章	Night Sight 的學習式白平衡;資料增強取代白平衡
HSV 顏色分割	第 9、10、12 章	分割的顏色依據;型態學清理;CamShift 追蹤
光源控制	第 7、9、17 章	AOI 取像設計;打光不均的處理;治本優於治標

表 8-2 第 8 章與全書其他章節的觀念連結

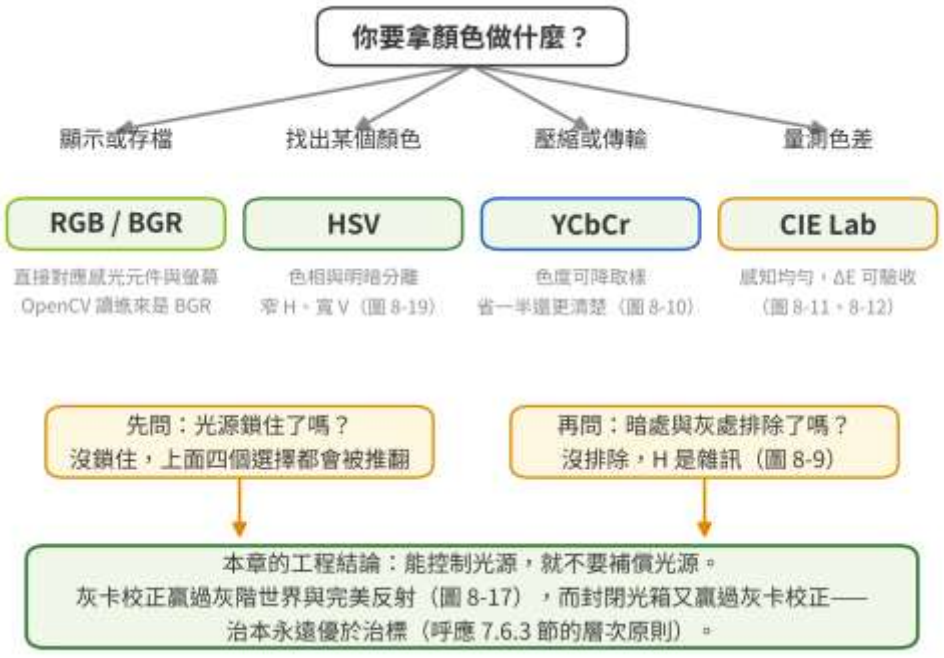


圖 8-21　第 8 章的決策地圖

程式實作 Lab 8

Lab 8-1 色彩空間探索器

目標：建立對色彩空間的直覺，並打造整學期最常用的調參工具。(a)寫一支程式，以 cv2.createTrackbar 建立六個滑桿(H、S、V 的上下界)，即時顯示遮罩與被遮罩後的結果;(b)用它找出教室中五種不同顏色物件的 HSV 範圍，製成表格;(c)驗證 8.1.3 節的論述：記錄同一物件在明亮處與陰影中的 RGB 與 HSV 值，比較哪些通道變化大、哪些穩定;(d)討論：為什麼顏色分割通常對 H 設窄門檻、對 V 設寬門檻?為什麼 S 與 V 必須設下限?(e)把這支工具存成共用程式，第 9、10、12 章的 Lab 都會用到。

Lab 8-2 洋裝效應實測

目標：親身驗證叫獸開講的現象。(a)取一個顏色明確的物件(如紅色馬克杯)，分別在窗邊日光、日光燈、白熾燈、以及手機的暖色螢幕光下拍攝;(b)量測同一位置的 RGB 與 HSV 值，製成表格，計算各通道的變化幅度;(c)套用灰階世界與完美反射兩種白平衡，比較校正後的數值是否收斂;(d)在畫面中放入一張白紙作為參考，以「白紙應為中性灰」為準則手動校正，比較與自動演算法的差異;(e)討論：這個實驗對「用顏色做工業檢測」有什麼啟示?

Lab 8-3 虛擬色彩與色階評估

目標：培養視覺化的專業素養。(a)取一張灰階影像(熱影像、深度圖或一般照片)，分別以 JET、HOT、VIRIDIS、BONE 四種色階顯示;(b)執行程式 8-2 的檢驗，量化各色階的亮度是否單調遞增，找出 JET 在哪個區段違反單調性;(c)把四張結果轉成黑白列印(或轉灰階)，比較哪些色階仍保有層次;(d)模擬色盲觀看：把結果的紅綠通道混合，觀察哪些色階仍可判讀;(e)結論：為一份要投稿的技術報告選擇色階，你會怎麼選?請說明理由。

Lab 8-4 循跡機器人的顏色偵測

目標：完成可上機的顏色追蹤模組。(a)以程式 8-4 為基礎，改用 ESP32-CAM 的串流網址作為輸入(第 2 章);(b)加入白平衡前處理，比較開啟與關閉時在不同光源下的穩定度;(c)輸出控制訊號：依目標物中心與畫面中心的偏差量，印出轉向指令與偏差角度;(d)量測系統的處理速度(FPS)，並思考：若改用第 16 章的 YOLO 偵測，速度與穩定度會如何變化?各適合什麼場合?(e)挑戰題：同時追蹤兩種顏色的物件並標示兩者的距離，或設計一個「顏色遙控」——用不同顏色的卡片對機器人下指令。

Lab 8-5 色溫對檢測的影響實測

目標：親手量出光源對顏色判讀的影響，並驗證白平衡的補救效果。(a)取三種不同色溫的光源（例如暖白燈泡、日光燈、手機手電筒），在同一位置拍攝同一組色卡或同一批水果，相機務必關閉自動白平衡與自動曝光，並記錄每次的快門與 ISO;(b)量測同一個色塊在三種光源下的 RGB 與 HSV，並以 skimage.color.deltaE_ciede2000 計算兩兩之間的色差;(c)分別套用灰階世界、完美反射與灰卡校正，重新計算色差，做出一張類似圖 8-17 的長條圖;(d)把其中一次拍攝換成「畫面中塞滿同一種顏色」的構圖，重做 (c)，觀察灰階世界是否如圖 8-16 所預測的那樣失效;(e)寫出 300 字的結論：若你是產線工程師，你會選哪一種策略，為什麼。

Lab 8-6 色階的專業檢驗工具

目標：做出一支可以重複使用的色階評估工具，培養視覺化的專業判斷力。(a)寫一個函式，輸入任一 OpenCV 或 matplotlib 的 colormap，輸出三項指標：L* 的下降次數、L* 每階變化率的最大值與最小值之比、以及綠色盲模擬後相距 16 階的最小色差;(b)綠色盲模擬請實作 Viénot, Brettel & Mollon (1999) 的方法——sRGB 先轉線性、再轉 LMS，令 M 通道由 L 與 S 線性重建，最後轉回 sRGB;(c)用這支工具評比 JET、HOT、VIRIDIS、BONE、TURBO 五種色階，做出一張類似圖 8-14 的對照表，並檢查你的結果是否與本書一致;(d)找一張你自己報告或論文裡用過的熱圖，用這支工具檢驗它的色階，若不合格就換掉並比較前後的可讀性;(e)進階：把工具包成命令列程式，加

入「黑白列印模擬」（轉灰階後看是否仍保有層次），放進你的個人工具箱。

本章重點整理

- 人眼三種錐狀細胞是三原色理論的生理基礎;同色異譜使不同光譜可呈現相同顏色，是彩色顯示成立的原因，也使一般 RGB 相機無法區分材質(需多光譜相機)。
- RGB 為加色系統(螢幕),CMY(K) 為減色系統(印刷)，色域不同;RGB 對機器友善但對人不直觀，且同一物體在明暗下三通道全變，不利顏色偵測。
- HSV/HSL 分離「色相」與「明暗鮮豔度」;幾何上可用圓柱、圓錐(HSV)、雙圓錐(HSL)詮釋，HSL 的純色位於 L=0.5 的赤道。色相對光照變化較穩健，故顏色分割以 H 為主要判準。
- OpenCV 的 HSV 範圍為 H:0~179、S/V:0~255;偵測紅色需跨 0 度接縫以兩段區間取聯集;暗處與灰處的 H 無意義，必須設定 S 與 V 的下限。
- YCbCr 為壓縮而生(色度可降取樣),CIE Lab 為感知均勻而生(可量測色差 ΔE);「把資源分配給人眼在意之處」的思想貫穿拜爾陣列、色度降取樣與 JPEG 量化。
- 虛擬色彩利用人眼辨色優於辨灰的特性;JET 因亮度非單調而可能製造假邊界且對色盲不友善，建議改用 VIRIDIS 等感知均勻色階。
- 白平衡是色彩恆常性的工程版本：灰階世界(假設場景平均為灰，單色調畫面失效)、完美反射(以最亮處為白，過曝時誤判)、標準色卡校正(最可靠)、學習式白平衡(現代路線)。工業場合應優先從源頭控制光源。

習題

1. 什麼是同色異譜?它為什麼是彩色顯示技術得以成立的基礎?它對機器視覺有什麼限制上的啟示?

提示與參考解答：同色異譜指兩束光譜完全不同的光，只要刺激三種錐狀細胞的比例相同，人眼就覺得是同一顏色。它讓螢幕只用紅綠藍三種光源就能模擬幾乎所有顏色。啟示：一般 RGB 相機與人眼一樣只記錄三個數值，因此無法分辨同色異譜的物體(如兩塊看似同色但材質不同的塑膠);要真正區分材質需使用記錄完整光譜的多光譜或高光譜相機。

2. 說明加色系統與減色系統的差異，並解釋為何螢幕上鮮豔的照片印出來常變暗淡。

提示與參考解答：加色系統(RGB)是光源相加，三色全開得白，用於自發光裝置(螢幕、投影機、LED)。減色系統(CMY)是顏料吸收特定波長，疊愈多愈暗，三色相加理論得黑(實務加 K 成 CMYK)，用於印刷。變暗淡的原因：兩系統的色域(可表現的顏色範圍)不同，螢幕能顯示的某些鮮豔顏色超出印刷色域，只能以最接近的顏色代替。

3. 為什麼說 RGB 不利於顏色偵測?請以「同一物體在陽光下與陰影中」說明，並解釋 HSV 如何改善。

提示與參考解答：同一顆紅蘋果在陽光下可能是 (220,40,40)、陰影中變成 (90,15,15)，三個通道都大幅改變，以 RGB 數值範圍偵測極不可靠。HSV 把「是什麼顏色」(H)與「有多亮」(V)分離：明暗變化主要影響 V，而 H 相當穩定。因此以 H 為主要判準、對 V 放寬門檻的分割策略，遠比在 RGB 空間設範圍穩健。

4. 請比較 HSV 的圓錐模型與 HSL 的雙圓錐模型，說明最飽和的純色分別位於幾何體的哪個位置，以及兩者各適合什麼應用。

提示與參考解答：HSV 圓錐：底部尖端為黑，頂面圓心為白、頂面外緣為最飽和的純色(白與純色同在 V=1)。HSL 雙圓錐：上下尖端分別為白與黑，最飽和的純色在中間高度 L=0.5 的赤道圓周。應用：HSV 適合顏色分割與追蹤(H 加 S 門檻直覺、V 便於放寬);HSL 適合調色盤與繪圖軟體(把滑桿拉到最上應為白色，符合使用者對「亮度」的期待)。

5. OpenCV 的色相 H 為什麼是 0 到 179 而非 0 到 359?偵測紅色時要如何處理?為什麼還必須設定 S 與 V 的下限?

提示與參考解答：因為要塞進 8 位元 uint8(上限 255)，把 0 至 360 度除以二成為 0 至 179。紅色的色相跨越 0 度接縫，須設兩段區間(如 0~10 與 170~179)再取聯

集。必須設 S 與 V 下限的原因：當顏色很暗或很不飽和(接近黑白灰)時，H 值極不穩定甚至無意義(圓錐尖端所有色相收縮成一點)，不排除這些區域會造成大量誤判。

6. YCbCr 的 4:2:0 降取樣為何幾乎不影響觀感?這個設計思想在本書還出現在哪兩個地方?

   提示與參考解答：因為人眼對亮度細節的敏感度遠高於色彩細節，把色度解析度砍成四分之一，人眼幾乎察覺不出。同樣的「把資源分配給人眼在意之處」思想還出現在：第 3 章的拜爾陣列讓綠色佔一半(人眼對綠光最敏感)、第 11 章 JPEG 對人眼不敏感的高頻係數用較大的量化步長。

7. 什麼是「感知均勻」?CIE Lab 為何需要它?為什麼對彩色影像做 CLAHE 時應轉到 Lab 只處理 L 通道?

   提示與參考解答：感知均勻指在該色彩空間中，兩顏色的歐氏距離大致正比於人眼感受到的色差大小。RGB 做不到——同樣的數值差距在不同區域造成的視覺差異天差地別。Lab 為此設計，使色差可量化(ΔE)，用於印刷紡織品管與色彩校正。做 CLAHE 只處理 L 通道的原因：若對 BGR 三通道分別等化，各通道映射函數不同，色彩比例被破壞而產生色偏;Lab 把亮度與色彩分離，只調 L 不動 a、b，色彩即可保持。

8. 虛擬色彩的目的是什麼?為什麼近年 JET 色階受到批評?請列出三個理由。

   提示與參考解答：目的：把沒有顏色的資料(溫度、深度、訊號強度)映射成彩色，利用人眼辨色能力遠強於辨灰(數萬種顏色 vs. 數十階灰度)的特性，讓細微差異一目了然。JET 受批評的三個理由：一、亮度非單調遞增(中間黃綠色比兩端亮)，會製造實際不存在的假邊界而誤導判讀;二、對色盲使用者不友善(約 8% 男性有紅綠色盲);三、轉黑白列印後資訊喪失。建議改用 VIRIDIS 等感知均勻色階。

9. 說明灰階世界白平衡的假設與失效情境。完美反射假設又在什麼情況下誤判?

   提示與參考解答：灰階世界假設整個場景的平均顏色應為中性灰，若實測偏藍就判定光源偏藍並反向校正。失效情境：整片草地、整面紅牆、大片藍天等單一色調畫面，假設不成立，演算法會把草地硬校正成灰綠色。完美反射假設以影像中最亮的像素為純白計算增益，在過曝或有高光反射時誤判——因為過曝像素已飽和，色彩資訊喪失，無法作為可靠的白色參考。

10. 顏色分割時，為什麼通常對 H 設定較窄的門檻，而對 V 設定較寬的門檻?

提示與參考解答：因為 H(色相)是「是什麼顏色」的主要判準，同一物體在不同光照下 H 相當穩定，設窄門檻可精確鎖定目標顏色、排除其他顏色。而 V(明度)會隨光照、陰影、角度大幅變化，設寬門檻才能容許這些變化不致漏檢。但 V(與 S)仍須設下限，以排除色相無意義的過暗與過灰區域。

11. 承叫獸開講：什麼是色彩恆常性?為什麼同一張洋裝照片會讓人看到截然不同的顏色?研究者用什麼方法驗證了這個解釋?

    提示與參考解答：色彩恆常性：大腦會自動推測當下的光源顏色並扣除其影響，還原物體本身的顏色(人腦內建的自動白平衡)。洋裝照片曝光過度、背景線索模糊，光源資訊極度曖昧，大腦只能依各自的先驗假設來猜：假設偏藍日光則扣掉藍色而看到白金，假設偏黃人造光則扣掉黃色而看到藍黑。驗證方法：研究團隊把洋裝影像嵌入光源線索明確的場景中，大多數人的判斷就趨於一致，且與被暗示的光源方向吻合。

12. (綜合設計題)請為一條農產品分級產線設計「不會被光源騙倒」的顏色檢測方案，列出至少四項具體措施並說明理由。

    提示與參考解答：一、控制光源：建立封閉檢測箱、使用高演色性且色溫穩定的 LED，隔絕環境光——從源頭消除變異，治本優於治標(呼應第 7.6.3 節的層次原則)。二、放置標準色卡：每次拍攝納入 ColorChecker，計算色彩校正矩陣，補償光源漂移與感測器差異。三、選對色彩空間：以 HSV 的 H 或 Lab 的 a、b 為判準，對明度放寬，降低光照敏感度。四、光學設計：調整光源方向與擴散以避免高光反射(反光處過曝會喪失色彩資訊)。五、定期驗證：LED 會隨使用時間衰減偏色，應定期以色卡檢查色彩響應是否漂移。六、備援策略：若無法完全控制光源(如田間)，則以第 15 章的資料增強讓模型適應光線變化。

13. OpenCV 的色相只有 0 到 179，而一個正紅色物體的色相中心正好落在 0。若你用 cv2.inRange(hsv, (0, 120, 60), (10, 255, 255)) 取遮罩，大約會抓到這個物體的多少比例?為什麼程式不會報錯?

    提示與參考解答：大約只會抓到一半。物體的色相會因為量測雜訊散布在 0 的兩側，而落在 170~179 的那一半完全不在區間內。圖 8-8 的實測是 50%，把門檻中心掃過 160 到 199 後最差為 49%。程式不會報錯，因為 inRange 的語意只是「在不在這個區間」，它不知道資料是環狀的;這類錯誤只會表現為遮罩莫名其妙地少一

塊，debug 時極容易被誤判成門檻設太緊。正解是改用兩段區間取聯集，或把色相先旋轉 90 再比對。

14. 為什麼顏色分割時一定要同時設定 S 與 V 的下限?請從 RGB 轉 HSV 的公式解釋，並估計在很暗或很不飽和的區域中，RGB 只變動一個量化階會造成多大的色相跳動。

    提示與參考解答：色相的三條公式分母都是 max(R,G,B) − min(R,G,B)，也就是飽和度的分子。顏色愈接近灰或愈暗，這個分母愈接近零，微小的輸入變動就被放大成巨大的角度變化（8.2.3 節的推導）。圖 8-9 的實測：對一張真實照片的每個像素加上 −1 到 +1 的隨機擾動，S<50 或 V<50 的區域平均跳動 2.4 度，其餘區域只跳 0.4 度;更極端的 S<25 或 V<25 則跳 3.9 度。在 S、V 都很小的角落，單一量化階就足以讓色相跳過數十度。

15. 一張 1920×1080 的彩色影像，若採用 4:2:0 色度降取樣，資料量會變成原來的幾分之幾?若反過來把亮度降取樣而保留完整色度，資料量又是多少?哪一種划算?為什麼?

    提示與參考解答：4:2:0 把 Cb、Cr 的解析度各砍成四分之一，資料量為 (1 + 0.25 + 0.25)/3 = 50%。反過來砍亮度則是 (0.25 + 1 + 1)/3 = 75%。降色度不但省得多（省 50% vs 省 25%），品質還更好：圖 8-10 在同一張照片上量到 39.7 dB 對 28.5 dB，相差 11.2 dB。原因是細節幾乎全部記在亮度通道裡——該圖量到 Y 的平均梯度為 55，而 Cb、Cr 只有 13 與 16，約為 Y 的三成;砍色度砍掉的多半是本來就不存在的東西。

16. 什麼叫「感知均勻」?請設計一個實驗來檢驗 RGB 空間是不是感知均勻的，並說明你會用什麼當作「人眼感受」的基準。

    提示與參考解答：感知均勻指空間中的歐氏距離正比於人眼感受到的色差。實驗設計：隨機取大量顏色對，令它們在 RGB 中的歐氏距離固定（例如都等於 20），再計算每一對的 ΔE2000（CIE 現行標準，可當作感知基準）。若 RGB 是感知均勻的，所有 ΔE 應該差不多;圖 8-11 的實測結果是 ΔE 從 0.4 一路散到 24，相差六十餘倍，所以 RGB 顯然不是。進一步可比較兩種量尺對 ΔE2000 的預測力：RGB 歐氏距離的 Spearman 相關為 0.75，Lab 歐氏距離為 0.88。注意 ΔE2000 本身也是模型而非真理，誠實的說法是「Lab 比 RGB 接近人眼」，不是「Lab 完美」。

17. 某紡織廠的驗收標準寫著「色差 ΔE 必須小於 2」。請說明這個數字在 RGB 上大約對應多少階的差異，以及為什麼這個標準不能改成「RGB 三通道各差不超過 5 階」。

    提示與參考解答：圖 8-12 沿著 Lab 空間的一個方向實測：ΔE 約 2 時 RGB 最大差約 6 階，ΔE 約 5 時約 16 階，ΔE 約 10 時約 31 階。但這個對應關係完全取決於顏色落在色空間的哪裡——同樣差 5 階，在某些顏色上 ΔE 不到 1，在另一些上卻超過 10（圖 8-11）。因此「RGB 差不超過 5 階」在不同色號上代表寬嚴差距極大的標準，無法作為統一的驗收門檻。這正是 CIE Lab 與 ΔE 存在的理由：它讓一個數字在所有顏色上代表同一件事。

18. 為什麼科學視覺化領域近年普遍不建議使用 JET 色階?請列出三個理由，並各自說明「要怎麼量」才能證明它。

    提示與參考解答：(一)亮度非單調。量法：把色階的 256 階轉成 CIE Lab 取 L，數 L* 下降的次數——圖 8-14 量到 JET 有 114 次，VIRIDIS 只有 2 次;更嚴重的後果是 JET 有相差 236 階的兩個數值感知亮度完全相同，黑白列印後它們會變成同一個灰。(二)假邊界。量法：看 L* 每階的變化率，JET 最快處是最慢處的 76 倍，VIRIDIS 只有 1.4 倍;變化率快的地方會讓平滑的資料看起來有一條邊。(三)對色盲不友善。量法：做綠色盲模擬後，計算相距 16 階的數值配對中最小的色差——JET 由 5.3 掉到 1.6（低於分辨門檻），VIRIDIS 只由 5.1 掉到 3.3。*

19. 請用普朗克黑體輻射與 CIE 配色函數，說明為什麼色溫從 6500 K 降到 2700 K 時，藍色通道的變化遠大於紅色通道。這對工業檢測的光源選用有什麼啟示?

    提示與參考解答：黑體輻射的光譜峰值隨溫度下降而往長波長移動（維因位移定律），因此低色溫光源在短波段的能量急遽減少。圖 8-15 實算的三通道增益顯示：由 6500 K 到 2700 K，紅色增益約變為 2.2 倍，而藍色增益掉到約 0.22 倍——藍色的相對變化大了一個量級。啟示：(一)檢測藍色或青色標記時，光源色溫的漂移影響最大，必須優先鎖定;(二)廉價 LED 的光譜不連續，其等效色溫會隨老化漂移，應選用高演色性、色溫規格明確的光源並定期以色卡複驗。

20. 灰階世界白平衡的輸出可以寫成 I(x)/mean(I)。請由此推導它在什麼條件下會給出正確答案，並解釋為什麼「單一色調的畫面會失效」這句話並不精確。

    提示與參考解答：設 I(x) = L ⊙ ρ(x)，則 mean(I) = L ⊙ mean(ρ)，故輸出為

ρ(x)/mean(ρ)——光源 L 一定會被完全消掉。因此正確與否只取決於 mean(ρ) 是否為中性灰;若不是，殘留的就是場景自身的色偏，與光源無關。圖 8-16 以 287 個真實影像區塊實測：「平均顏色偏離中性灰的程度」與誤差的 Spearman 相關高達 0.94，而「色相的環狀離散度」只有 −0.34。單一色調之所以常出事，是因為它通常同時帶著強烈色偏，而不是因為色相集中本身有問題;一張色相集中卻接近中性的照片（如淡藍的雪景）其實不會失效。

21. 在圖 8-17 的實測中，灰階世界在暖色主導的場景裡誤差高達 ΔE 21，比完全不做白平衡的 ΔE 4.7 還糟。請解釋為什麼會這樣，並說明這對「要不要在產線上開啟自動白平衡」的決策有何意義。

提示與參考解答：因為灰階世界的誤差來自場景自身的色偏（上一題的推導），而不是來自光源;場景愈偏離中性，它造成的傷害愈大。在暖色主導的場景中，它把「這個場景本來就偏橘」誤判成「光源偏橘」，於是硬把木頭校成了灰。決策意義：自動白平衡是一個「做了假設」的演算法，假設不成立時它會主動製造誤差;在場景內容固定且偏離中性的產線上（例如全部都是紅色番茄），應該關閉自動白平衡，改用灰卡或色卡做一次性的固定校正，並把增益參數鎖死。圖 8-17 中灰卡校正在兩種場景都維持在 ΔE 1.5 以內。

22. 為什麼顏色分割的實務心法是「對 H 設窄門檻、對 V 設寬門檻」?如果有人反過來做，會發生什麼事?請用可量化的方式說明。

提示與參考解答：H 是「是什麼顏色」的主要判準且對光照相對穩定，放寬會把鄰近色相的干擾物一起抓進來;V 會隨光照、陰影、角度大幅變化，設窄會把陰影裡的目標切掉。圖 8-19 在一個已知正確答案的合成場景上掃過所有門檻組合：IoU 在最佳值 97% 以內的「可用區」中，H 半寬只能落在 ±7 到 ±10 之間（約佔掃描範圍的一成），V 的下限卻從 10 到 90 都行（約佔四成）。越界的代價：V 下限拉到 150，陰影裡的目標被切掉，IoU 由 0.94 掉到 0.37;H 放寬到 ±40，干擾物被抓進來，IoU 掉到 0.57。

23. 圖 8-18 的第五步「取最大輪廓並填滿」之後，像素數反而由 57,242 增加到 76,324。這是程式寫錯了嗎?請說明原因，並討論什麼情況下不該做這一步。

提示與參考解答：不是錯誤。目標（紅色托盤）是一個環形，填滿外輪廓等於把中間那只杯子一併算進遮罩，所以面積會變大。要不要填，完全取決於下游任務：若

目的是取得「整組杯盤」的位置與方框以便機械手抓取，填滿是正確的;若目的是量測托盤本身的面積或判斷環形是否完整（例如檢測墊圈的缺口），填滿就會把你要找的缺陷抹掉，必須改用 RETR_CCOMP 或 RETR_TREE 保留內輪廓。這題的要點是：形態學與輪廓處理沒有通用的「正確做法」，只有「與下游任務一致的做法」。

24. 一條番茄分級產線以色相劃分三個等級（界線在 20、40、60 度）。若工廠的照明從 6500 K 換成 2700 K，大約會有多少比例的樣本被改判?請提出兩種補救方案並比較其效果。

提示與參考解答：圖 8-20 的實測顯示，60 顆樣本中有 32 顆（約 53%）被改判——因為量到的色相整條平移了。補救方案一是灰階世界：它把 2700 K 的錯誤由 32 顆降到 18 顆，但在光源本來就正確的 6500 K 反而無中生有製造出 20 顆錯誤（因為番茄色帶本身就是強烈色偏，踩中圖 8-16 的失效條件）。方案二是在畫面中放一張灰卡：錯誤由 32 顆降到 4 顆，且在其餘色溫下都是 0 到 2 顆。結論是方案二明顯較佳;但最佳解其實是第三種——建立封閉光箱把光源鎖死，讓問題根本不發生（呼應 7.6.3 節的層次原則）。

25. (綜合設計題)你要為一條電子零件產線設計色碼辨識系統，需分辨十種顏色的線材，允許的誤判率低於萬分之一。請提出完整的設計，涵蓋光源、色彩空間、門檻策略、校正程序與驗收指標，並說明每一項的理由與對應的量化依據。

提示與參考解答：一、光源：封閉光箱、高演色性 LED、色溫規格明確並記錄批號;理由是圖 8-15 顯示只換光源就能造成 ΔE 10 以上的偏移，而圖 8-20 顯示這足以改判過半樣本。二、校正：每批開工前以標準色卡拍一張，計算並鎖定色彩校正矩陣，關閉相機的自動白平衡與自動曝光;理由是圖 8-17 顯示灰卡校正是唯一在不同場景都穩定的方法（ΔE 1.5 以內），而自動白平衡在色偏場景會主動製造誤差。三、色彩空間：量測與判別用 CIE Lab（感知均勻，ΔE 可直接當門檻），即時分割用 HSV（運算快、門檻直覺）。四、門檻策略：窄 H 寬 V，V 下限只用來排除色相無意義的暗處（圖 8-9、8-19）;十種顏色應先量測彼此的 ΔE，確認最接近的一對也大於 5，否則要求更換線材色號。五、驗收指標：以每種顏色各 1000 個樣本建立混淆矩陣，要求最小類間 ΔE、類內 ΔE 標準差、以及跨班別（早中晚班）的重複性;萬分之一的誤判率需要類間距離至少為類內標準差的四倍以上。六、長期監控：每日

記錄色卡的量測值，一旦漂移超過 ΔE 2 即觸發光源更換——預防永遠比補償便宜。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 6(彩色影像處理)。
- Lafer-Sousa, R., Hermann, K. L., & Conway, B. R., Striking Individual Differences in Color Perception Uncovered by "the Dress" Photograph, Current Biology, 25(13), 2015.(本章叫獸開講之科學來源)
- Wallisch, P., Illumination Assumptions Account for Individual Differences in the Perceptual Interpretation of a Profoundly Ambiguous Stimulus, Journal of Vision, 17(4), 2017.(逾一萬三千人的雲雀/夜貓子研究)
- Fairchild, M. D., Color Appearance Models, 3rd ed., Wiley, 2013.
- Ebner, M., Color Constancy, Wiley, 2007.(白平衡演算法之專書)
- OpenCV 官方文件：Color conversions(cv2.cvtColor 完整轉換代碼表)。
- 補充教材與課程資訊：@prof(https://page.line.me/prof)